

BACSE344 — Cloud Infrastructure and Architecture

Digital Assignment Source Code Compendium: **TO-DO-TRACKER**

Student Name	ANIKET MOHANTY
Registration Number	25BCE5816
Course Code & Title	BACSE344 — Cloud Infrastructure and Architecture
Slot	C1
Faculty In-Charge	Dr. P. Anandan
Frontend Production URL	https://taskflow-alpha-dun.vercel.app
Backend API URL	https://taskflow-backend-9huw.onrender.com
Database	MongoDB Atlas (AWS ap-south-1 Mumbai)
Source Code Repository	https://github.com/aniketmohanty04/taskflow

Table of Contents — Project Source Files

#	Component Description	File Path	Size
1	Backend - Server & Config	backend/server.js	98 lines
2	Backend - Authentication Middleware	backend/middleware/auth.js	22 lines

#	Component Description	File Path	Size
3	Backend - Error Handling Middleware	backend/middleware/errorHandler.js	51 lines
4	Backend Model - AuthUser	backend/models/AuthUser.js	49 lines
5	Backend Model - HabitTask	backend/models/HabitTask.js	50 lines
6	Backend Model - DailyCompletion	backend/models/DailyCompletion.js	33 lines
7	Backend Model - Task	backend/models/Task.js	108 lines
8	Backend Route - Authentication	backend/routes/auth.js	86 lines
9	Backend Route - Habit Tracking	backend/routes/habits.js	89 lines
10	Backend Route - Daily Completions & Chart	backend/routes/completions.js	132 lines
11	Backend Route - Tasks & Kanban	backend/routes/tasks.js	315 lines
12	Frontend - App Root Component	frontend/src/App.js	199 lines
13	Frontend - Navigation Bar	frontend/src/components/Navbar.js	57 lines
14	Frontend - Authentication Screen	frontend/src/components/LoginPage.js	122 lines
15	Frontend - Habit Tracker Grid	frontend/src/components/HabitTracker.js	282 lines
16	Frontend - Habit Task Modal Form	frontend/src/components/HabitTaskForm.js	116 lines

#	Component Description	File Path	Size
17	Frontend - 30-Day SVG Progress Chart	frontend/src/components/ProgressChart.js	167 lines
18	Frontend - Dashboard Metrics	frontend/src/components/Dashboard.js	80 lines
19	Frontend - Task Form Modal	frontend/src/components/TaskForm.js	211 lines
20	Frontend - Task List View	frontend/src/components/TaskList.js	257 lines
21	Frontend - Task Card Component	frontend/src/components/TaskCard.js	91 lines
22	Frontend Service - Auth API	frontend/src/services/authAPI.js	20 lines
23	Frontend Service - Habit API	frontend/src/services/habitAPI.js	50 lines
24	Frontend Service - Task API	frontend/src/services/api.js	82 lines
25	Frontend - Theme & Global Styles	frontend/src/App.css	624 lines
26	Frontend - PWA Web Manifest	frontend/public/manifest.json	35 lines
27	Frontend - Package Configuration	frontend/package.json	35 lines
28	Backend - Package Configuration	backend/package.json	33 lines

Complete Source Code Listings

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');
require('dotenv').config();

const taskRoutes = require('./routes/tasks');
const userRoutes = require('./routes/users');
const habitRoutes = require('./routes/habits');
const completionRoutes = require('./routes/completions');
const authRoutes = require('./routes/auth');
const errorHandler = require('./middleware/errorHandler');

const app = express();

// — Security & Middleware —————
app.use(helmet());
app.use(morgan('combined'));
app.use(cors({
  origin: [
    process.env.FRONTEND_URL || 'http://localhost:3000',
    'http://localhost:3000',
    'http://localhost:3001',
    'https://taskflow-alpha-dun.vercel.app',
    'https://taskflow-frontend.vercel.app',
    /\.vercel\.app$/ // allow any vercel preview URL
  ],
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials: true
}));
app.use(express.json({ limit: '10kb' }));
app.use(express.urlencoded({ extended: true }));

// — Database Connection —————
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGODB_URI);
    console.log(`✅ MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`❌ MongoDB Connection Error:`, error.message);
    process.exit(1);
  }
};

connectDB();
```

```

// — Routes —————
// Health Check Endpoint
app.get('/', (req, res) => {
  res.json({
    success: true,
    message: 'TaskFlow API is running 🚀',
    version: '1.0.0',
    timestamp: new Date().toISOString(),
    environment: process.env.NODE_ENV || 'development'
  });
});

app.get('/api/health', (req, res) => {
  res.json({
    success: true,
    status: 'healthy',
    database: mongoose.connection.readyState === 1 ? 'connected' : 'disconnected',
    uptime: process.uptime(),
    timestamp: new Date().toISOString()
  });
});

// API Routes
app.use('/api/auth', authRoutes);
app.use('/api/tasks', taskRoutes);
app.use('/api/users', userRoutes);
app.use('/api/habits', habitRoutes);
app.use('/api/completions', completionRoutes);

// — 404 Handler —————
app.use((req, res, next) => {
  res.status(404).json({
    success: false,
    message: `Route ${req.originalUrl} not found`
  });
});

// — Global Error Handler —————
app.use(errorHandler);

// — Start Server —————
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`🚀 TaskFlow Server running on port ${PORT}`);
  console.log(`📍 Environment: ${process.env.NODE_ENV || 'development'}`);
  console.log(`🌐 API Base URL: http://localhost:${PORT}/api`);
});

module.exports = app;

```

```
const jwt = require('jsonwebtoken');

const JWT_SECRET = process.env.JWT_SECRET || 'taskflow_super_secret_jwt_key_2026';

const auth = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ success: false, message: 'Access denied. No authentication token provided.' });
  }

  const token = authHeader.split(' ')[1];
  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.user = { id: decoded.id };
    next();
  } catch (err) {
    return res.status(401).json({ success: false, message: 'Invalid or expired authentication token.' });
  }
};

module.exports = auth;
```

```
/**
 * Global Error Handling Middleware
 * Catches all errors thrown in route handlers and sends a uniform JSON response
 */
const errorHandler = (err, req, res, next) => {
  let statusCode = err.statusCode || 500;
  let message = err.message || 'Internal Server Error';

  // Mongoose Bad ObjectId
  if (err.name === 'CastError') {
    statusCode = 400;
    message = `Invalid ${err.path}: ${err.value}`;
  }

  // Mongoose Duplicate Key Error
  if (err.code === 11000) {
    statusCode = 409;
    const field = Object.keys(err.keyValue)[0];
    message = `Duplicate value for field: ${field}. Please use a different value.`;
  }

  // Mongoose Validation Error
  if (err.name === 'ValidationError') {
    statusCode = 400;
    message = Object.values(err.errors).map(val => val.message).join(', ');
  }

  // JWT Errors (if auth added later)
  if (err.name === 'JsonWebTokenError') {
    statusCode = 401;
    message = 'Invalid token. Please log in again.';
  }
  if (err.name === 'TokenExpiredError') {
    statusCode = 401;
    message = 'Token expired. Please log in again.';
  }

  // Log error in development
  if (process.env.NODE_ENV === 'development') {
    console.error('🔴 Error:', err);
  }

  res.status(statusCode).json({
    success: false,
    message,
    ...(process.env.NODE_ENV === 'development' && { stack: err.stack })
  });
};
```

```
module.exports = errorHandler;
```

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const authUserSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true,
      minlength: 2,
      maxlength: 50
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Invalid email format']
    },
    password: {
      type: String,
      required: [true, 'Password is required'],
      minlength: 6,
      select: false // never returned in queries by default
    },
    avatar: {
      type: String,
      default: ''
    }
  },
  { timestamps: true }
);

// Hash password before save
authUserSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

// Compare password method
authUserSchema.methods.comparePassword = async function (candidatePassword) {
  return bcrypt.compare(candidatePassword, this.password);
};
```

```
const AuthUser = mongoose.model('AuthUser', authUserSchema);  
module.exports = AuthUser;
```

```
const mongoose = require('mongoose');

const TASK_COLORS = ['#818cf8', '#60a5fa', '#34d399', '#f472b6', '#fbbf24', '#f97316', '#2dd4bf', '#a78bfa'];

const habitTaskSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Task name is required'],
      trim: true,
      minlength: [2, 'Name must be at least 2 characters'],
      maxlength: [80, 'Name cannot exceed 80 characters']
    },
    type: {
      type: String,
      enum: ['fixed', 'variable'],
      default: 'fixed'
    },
    // For variable tasks: 0=Sun, 1=Mon, 2=Tue, 3=Wed, 4=Thu, 5=Fri, 6=Sat
    scheduledDays: {
      type: [Number],
      default: []
    },
    color: {
      type: String,
      default: '#818cf8'
    },
    order: {
      type: Number,
      default: 0
    },
    userId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'AuthUser',
      required: true,
      index: true
    },
    isActive: {
      type: Boolean,
      default: true
    }
  },
  { timestamps: true }
);

habitTaskSchema.index({ userId: 1, order: 1 });
```

```
const HabitTask = mongoose.model('HabitTask', habitTaskSchema);
module.exports = HabitTask;
```

6 Backend Model - DailyCompletion

backend/models/DailyCompletion.js

```
const mongoose = require('mongoose');

const dailyCompletionSchema = new mongoose.Schema(
  {
    taskId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'HabitTask',
      required: true
    },
    date: {
      type: String, // stored as "YYYY-MM-DD"
      required: true
    },
    completed: {
      type: Boolean,
      default: true
    },
    userId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'AuthUser',
      index: true
    }
  },
  { timestamps: true }
);

// Unique per task per day
dailyCompletionSchema.index({ taskId: 1, date: 1 }, { unique: true });
dailyCompletionSchema.index({ userId: 1, date: 1 });

const DailyCompletion = mongoose.model('DailyCompletion', dailyCompletionSchema);
module.exports = DailyCompletion;
```

```
const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, 'Task title is required'],
      trim: true,
      minlength: [3, 'Title must be at least 3 characters'],
      maxlength: [100, 'Title cannot exceed 100 characters']
    },
    description: {
      type: String,
      trim: true,
      maxlength: [500, 'Description cannot exceed 500 characters'],
      default: ''
    },
    status: {
      type: String,
      enum: {
        values: ['todo', 'in-progress', 'completed'],
        message: 'Status must be todo, in-progress, or completed'
      },
      default: 'todo'
    },
    priority: {
      type: String,
      enum: {
        values: ['low', 'medium', 'high'],
        message: 'Priority must be low, medium, or high'
      },
      default: 'medium'
    },
    category: {
      type: String,
      trim: true,
      maxlength: [50, 'Category cannot exceed 50 characters'],
      default: 'General'
    },
    dueDate: {
      type: Date,
      default: null
    },
    assignedTo: {
      type: String,
      trim: true,
      maxlength: [50, 'Assigned to cannot exceed 50 characters'],
      default: ''
    }
  }
);
```

```

    },
    tags: {
      type: [String],
      default: []
    },
    completedAt: {
      type: Date,
      default: null
    },
    userId: {
      type: String,
      default: 'anonymous'
    }
  },
  {
    timestamps: true, // Adds createdAt and updatedAt automatically
    toJSON: { virtuals: true },
    toObject: { virtuals: true }
  }
);

```

```

// — Indexes for performance —————
taskSchema.index({ status: 1 });
taskSchema.index({ priority: 1 });
taskSchema.index({ userId: 1 });
taskSchema.index({ createdAt: -1 });

```

```

// — Virtual: isOverdue —————
taskSchema.virtual('isOverdue').get(function () {
  if (!this.dueDate || this.status === 'completed') return false;
  return new Date() > new Date(this.dueDate);
});

```

```

// — Pre-save Hook —————
taskSchema.pre('save', function (next) {
  if (this.status === 'completed' && !this.completedAt) {
    this.completedAt = new Date();
  }
  if (this.status !== 'completed') {
    this.completedAt = null;
  }
  next();
});

```

```

// — Static Methods —————
taskSchema.statics.getStats = async function () {
  const stats = await this.aggregate([
    {
      $group: {
        _id: '$status',
        count: { $sum: 1 }
      }
    }
  ]);

```

```
    }  
  }  
  ]);  
  return stats;  
};  
  
const Task = mongoose.model('Task', taskSchema);  
module.exports = Task;
```

```
const express = require('express');
const router = express.Router();
const jwt = require('jsonwebtoken');
const { body, validationResult } = require('express-validator');
const AuthUser = require('../models/AuthUser');

const JWT_SECRET = process.env.JWT_SECRET || 'taskflow_super_secret_jwt_key_2026';
const JWT_EXPIRES = '7d';

function signToken(id) {
  return jwt.sign({ id }, JWT_SECRET, { expiresIn: JWT_EXPIRES });
}

function handleErrors(req, res, next) {
  const errors = validationResult(req);
  if (!errors.isEmpty())
    return res.status(400).json({ success: false, message: errors.array()[0].msg });
  next();
}

// — POST /api/auth/register —————
router.post('/register', [
  body('name').trim().notEmpty().withMessage('Name is required').isLength({ min: 2 }),
  body('email').isEmail().withMessage('Valid email required').normalizeEmail(),
  body('password').isLength({ min: 6 }).withMessage('Password must be at least 6 characters'),
], handleErrors, async (req, res, next) => {
  try {
    const { name, email, password } = req.body;
    const existing = await AuthUser.findOne({ email });
    if (existing)
      return res.status(400).json({ success: false, message: 'Email already registered' });

    const user = await AuthUser.create({ name, email, password });
    const token = signToken(user._id);

    res.status(201).json({
      success: true,
      message: 'Account created successfully!',
      token,
      user: { id: user._id, name: user.name, email: user.email }
    });
  } catch (err) { next(err); }
});

// — POST /api/auth/login —————
router.post('/login', [
```

```

    body('email').isEmail().withMessage('Valid email required').normalizeEmail(),
    body('password').notEmpty().withMessage('Password is required'),
  ], handleErrors, async (req, res, next) => {
    try {
      const { email, password } = req.body;
      const user = await AuthUser.findOne({ email }).select('+password');
      if (!user || !(await user.comparePassword(password)))
        return res.status(401).json({ success: false, message: 'Invalid email or password'
    });

    const token = signToken(user._id);
    res.json({
      success: true,
      message: 'Login successful!',
      token,
      user: { id: user._id, name: user.name, email: user.email }
    });
  } catch (err) { next(err); }
});

// — GET /api/auth/me —————
router.get('/me', async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer '))
      return res.status(401).json({ success: false, message: 'Not authenticated' });

    const token = authHeader.split(' ')[1];
    const decoded = jwt.verify(token, JWT_SECRET);
    const user = await AuthUser.findById(decoded.id);
    if (!user)
      return res.status(401).json({ success: false, message: 'User not found' });

    res.json({ success: true, user: { id: user._id, name: user.name, email: user.email }
  });
  } catch (err) {
    res.status(401).json({ success: false, message: 'Invalid or expired token' });
  }
});

module.exports = router;

```

```
const express = require('express');
const router = express.Router();
const { body, validationResult } = require('express-validator');
const HabitTask = require('../models/HabitTask');
const auth = require('../middleware/auth');

// Protect all habit routes
router.use(auth);

const validate = [
  body('name').trim().notEmpty().withMessage('Name required').isLength({ min: 2, max: 80 }),
  body('type').optional().isIn(['fixed', 'variable']),
  body('scheduledDays').optional().isArray(),
  body('color').optional().isString(),
];

const handleErrors = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) return res.status(400).json({ success: false, errors: errors.array() });
  next();
};

// GET all active habit tasks for the authenticated user
router.get('/', async (req, res, next) => {
  try {
    const tasks = await HabitTask.find({ userId: req.user.id, isActive: true })
      .sort({ order: 1, createdAt: 1 });
    res.json({ success: true, data: tasks });
  } catch (err) { next(err); }
});

// POST create habit task for the authenticated user
router.post('/', validate, handleErrors, async (req, res, next) => {
  try {
    const { name, type, scheduledDays, color, order } = req.body;
    const count = await HabitTask.countDocuments({ userId: req.user.id, isActive: true });
    const task = await HabitTask.create({
      name,
      type,
      scheduledDays: scheduledDays || [],
      color,
      order: order ?? count,
      userId: req.user.id
    });
    res.status(201).json({ success: true, message: 'Habit task created', data: task });
  } catch (err) { next(err); }
});
```

```

    } catch (err) { next(err); }
  });

// PUT update habit task (user-isolated)
router.put('/:id', validate, handleErrors, async (req, res, next) => {
  try {
    const task = await HabitTask.findOneAndUpdate(
      { _id: req.params.id, userId: req.user.id },
      req.body,
      { new: true, runValidators: true }
    );
    if (!task) return res.status(404).json({ success: false, message: 'Task not found'
  });
    res.json({ success: true, message: 'Habit task updated', data: task });
  } catch (err) { next(err); }
});

// PATCH reorder habit task (user-isolated)
router.patch('/:id/order', async (req, res, next) => {
  try {
    const task = await HabitTask.findOneAndUpdate(
      { _id: req.params.id, userId: req.user.id },
      { order: req.body.order },
      { new: true }
    );
    if (!task) return res.status(404).json({ success: false, message: 'Task not found'
  });
    res.json({ success: true, data: task });
  } catch (err) { next(err); }
});

// DELETE habit task (soft delete, user-isolated)
router.delete('/:id', async (req, res, next) => {
  try {
    const task = await HabitTask.findOneAndUpdate(
      { _id: req.params.id, userId: req.user.id },
      { isActive: false },
      { new: true }
    );
    if (!task) return res.status(404).json({ success: false, message: 'Task not found'
  });
    res.json({ success: true, message: 'Habit task deleted', data: { deletedId: req.params.id } });
  } catch (err) { next(err); }
});

module.exports = router;

```

```
const express = require('express');
const router = express.Router();
const DailyCompletion = require('../models/DailyCompletion');
const HabitTask = require('../models/HabitTask');
const auth = require('../middleware/auth');

// Protect all completion routes
router.use(auth);

// GET /api/completions?month=YYYY-MM - all completions for a month for the user's habits
router.get('/', async (req, res, next) => {
  try {
    const { month } = req.query; // e.g. "2026-09"
    if (!month) return res.status(400).json({ success: false, message: 'month query required (YYYY-MM)' });

    // Get all task IDs belonging to the authenticated user
    const userTasks = await HabitTask.find({ userId: req.user.id }).select('_id').lean();
    const taskIds = userTasks.map(t => t._id);

    const completions = await DailyCompletion.find({
      taskId: { $in: taskIds },
      date: { $regex: `^${month}` },
      completed: true
    }).lean();

    res.json({ success: true, data: completions });
  } catch (err) { next(err); }
});

// GET /api/completions/chart?days=30 - daily completion % for chart for user's habits only
router.get('/chart', async (req, res, next) => {
  try {
    const days = Math.min(parseInt(req.query.days) || 30, 90);
    // Find ONLY the authenticated user's active tasks
    const tasks = await HabitTask.find({ userId: req.user.id, isActive: true }).lean();

    const result = [];
    const today = new Date();

    for (let i = days - 1; i >= 0; i--) {
      const d = new Date(today);
      d.setDate(today.getDate() - i);
      const dateStr = d.toISOString().split('T')[0];
      const dayOfWeek = d.getDay(); // 0=Sun...6=Sat
```

```

// Determine scheduled tasks for this day
const scheduled = tasks.filter(t =>
  t.type === 'fixed' ||
  (t.type === 'variable' && t.scheduledDays.includes(dayOfWeek))
);

if (scheduled.length === 0) {
  result.push({ date: dateStr, pct: 0, completed: 0, total: 0 });
  continue;
}

const completions = await DailyCompletion.countDocuments({
  taskId: { $in: scheduled.map(t => t._id) },
  date: dateStr,
  completed: true
});

result.push({
  date: dateStr,
  pct: Math.round((completions / scheduled.length) * 100),
  completed: completions,
  total: scheduled.length
});
}

res.json({ success: true, data: result });
} catch (err) { next(err); }
});

// POST /api/completions/toggle - toggle a task completion for a date
router.post('/toggle', async (req, res, next) => {
  try {
    const { taskId, date } = req.body;
    if (!taskId || !date) return res.status(400).json({ success: false, message: 'taskId
and date required' });

    // Verify task ownership
    const task = await HabitTask.findOne({ _id: taskId, userId: req.user.id });
    if (!task) return res.status(404).json({ success: false, message: 'Habit task not fo
und or access denied' });

    const existing = await DailyCompletion.findOne({ taskId, date });
    if (existing) {
      // Toggle
      existing.completed = !existing.completed;
      if (!existing.userId) existing.userId = req.user.id;
      await existing.save();
      res.json({ success: true, data: existing });
    } else {
      // Create new completion with userId

```

```

    const completion = await DailyCompletion.create({
      taskId,
      date,
      completed: true,
      userId: req.user.id
    });
    res.status(201).json({ success: true, data: completion });
  }
} catch (err) { next(err); }
});

// GET /api/completions/streak/:taskId - streak count for a task
router.get('/streak/:taskId', async (req, res, next) => {
  try {
    // Verify task ownership
    const task = await HabitTask.findOne({ _id: req.params.taskId, userId: req.user.id
  });
    if (!task) return res.status(404).json({ success: false, message: 'Habit task not found' });

    let streak = 0;
    let d = new Date();

    while (true) {
      const dateStr = d.toISOString().split('T')[0];
      const comp = await DailyCompletion.findOne({
        taskId: req.params.taskId,
        date: dateStr,
        completed: true
      });
      if (!comp) break;
      streak++;
      d.setDate(d.getDate() - 1);
    }

    res.json({ success: true, data: { streak } });
  } catch (err) { next(err); }
});

module.exports = router;

```

```
const express = require('express');
const router = express.Router();
const { body, validationResult } = require('express-validator');
const Task = require('../models/Task');
const auth = require('../middleware/auth');

// Protect all task routes with JWT auth
router.use(auth);

// — Validation Middleware —————
const validateTask = [
  body('title')
    .trim()
    .notEmpty().withMessage('Title is required')
    .isLength({ min: 3, max: 100 }).withMessage('Title must be 3-100 characters'),
  body('description')
    .optional()
    .trim()
    .isLength({ max: 500 }).withMessage('Description cannot exceed 500 characters'),
  body('status')
    .optional()
    .isIn(['todo', 'in-progress', 'completed']).withMessage('Invalid status value'),
  body('priority')
    .optional()
    .isIn(['low', 'medium', 'high']).withMessage('Invalid priority value'),
  body('category')
    .optional()
    .trim()
    .isLength({ max: 50 }).withMessage('Category cannot exceed 50 characters'),
  body('dueDate')
    .optional({ nullable: true, checkFalsy: true })
    .isISO8601().withMessage('Invalid date format'),
  body('assignedTo')
    .optional()
    .trim()
    .isLength({ max: 50 }).withMessage('AssignedTo cannot exceed 50 characters'),
  body('tags')
    .optional()
    .isArray().withMessage('Tags must be an array'),
];

const handleValidationErrors = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({
      success: false,
      message: 'Validation failed',
      errors: errors.array().map(err => ({ field: err.path, message: err.msg }))
    });
  }
  next();
};
```

```

    });
  }
  next();
};

// -----
// GET /api/tasks – Fetch all tasks for the logged in user with filters & paging
// -----
router.get('/', async (req, res, next) => {
  try {
    const {
      status,
      priority,
      category,
      search,
      page = 1,
      limit = 20,
      sortBy = 'createdAt',
      order = 'desc'
    } = req.query;

    // Filter by user ID first
    const filter = { userId: req.user.id };
    if (status) filter.status = status;
    if (priority) filter.priority = priority;
    if (category) filter.category = { $regex: category, $options: 'i' };
    if (search) {
      filter.$or = [
        { title: { $regex: search, $options: 'i' } },
        { description: { $regex: search, $options: 'i' } },
        { tags: { $in: [new RegExp(search, 'i')] } }
      ];
    }
  }

  // Pagination
  const pageNum = Math.max(1, parseInt(page));
  const limitNum = Math.min(100, Math.max(1, parseInt(limit)));
  const skip = (pageNum - 1) * limitNum;

  // Sort
  const sortOrder = order === 'asc' ? 1 : -1;
  const sortObj = { [sortBy]: sortOrder };

  // Execute query
  const [tasks, totalCount] = await Promise.all([
    Task.find(filter).sort(sortObj).skip(skip).limit(limitNum).lean(),
    Task.countDocuments(filter)
  ]);

  res.json({
    success: true,

```

```

    data: tasks,
    pagination: {
      currentPage: pageNum,
      totalPages: Math.ceil(totalCount / limitNum),
      totalCount,
      limit: limitNum,
      hasNextPage: pageNum < Math.ceil(totalCount / limitNum),
      hasPrevPage: pageNum > 1
    }
  });
} catch (error) {
  next(error);
}
});

// -----
// GET /api/tasks/stats – Dashboard statistics for the logged in user
// -----
router.get('/stats', async (req, res, next) => {
  try {
    const [statusStats, priorityStats, totalCount, overdueCount] = await Promise.all([
      Task.aggregate([
        { $match: { userId: req.user.id } },
        { $group: { _id: '$status', count: { $sum: 1 } } }
      ]),
      Task.aggregate([
        { $match: { userId: req.user.id } },
        { $group: { _id: '$priority', count: { $sum: 1 } } }
      ]),
      Task.countDocuments({ userId: req.user.id }),
      Task.countDocuments({
        userId: req.user.id,
        status: { $ne: 'completed' },
        dueDate: { $lt: new Date() }
      })
    ]);

    // Reshape into readable objects
    const byStatus = { todo: 0, 'in-progress': 0, completed: 0 };
    statusStats.forEach(s => { byStatus[s._id] = s.count; });

    const byPriority = { low: 0, medium: 0, high: 0 };
    priorityStats.forEach(p => { byPriority[p._id] = p.count; });

    res.json({
      success: true,
      data: {
        total: totalCount,
        overdue: overdueCount,
        byStatus,
        byPriority,

```

```

        completionRate: totalCount > 0
          ? ((byStatus.completed / totalCount) * 100).toFixed(1)
          : '0.0'
      }
    });
  } catch (error) {
    next(error);
  }
});

// -----
// GET /api/tasks/:id - Fetch single task for the logged in user
// -----
router.get('/:id', async (req, res, next) => {
  try {
    const task = await Task.findOne({ _id: req.params.id, userId: req.user.id });
    if (!task) {
      return res.status(404).json({
        success: false,
        message: 'Task not found'
      });
    }
    res.json({ success: true, data: task });
  } catch (error) {
    if (error.name === 'CastError') {
      return res.status(400).json({ success: false, message: 'Invalid task ID format'
});
    }
    next(error);
  }
});

// -----
// POST /api/tasks - Create a new task tied to the logged in user
// -----
router.post('/', validateTask, handleValidationErrors, async (req, res, next) => {
  try {
    const { title, description, status, priority, category, dueDate, assignedTo, tags }
= req.body;

    const task = await Task.create({
      title,
      description,
      status,
      priority,
      category,
      dueDate: dueDate || null,
      assignedTo,
      tags: tags || [],
      userId: req.user.id
    });

```

```

    res.status(201).json({
      success: true,
      message: 'Task created successfully',
      data: task
    });
  } catch (error) {
    next(error);
  }
});

// -----
// PUT /api/tasks/:id - Full update of a task (user-isolated)
// -----
router.put('/:id', validateTask, handleValidationErrors, async (req, res, next) => {
  try {
    const task = await Task.findOneAndUpdate(
      { _id: req.params.id, userId: req.user.id },
      { ...req.body },
      { new: true, runValidators: true }
    );

    if (!task) {
      return res.status(404).json({ success: false, message: 'Task not found' });
    }

    res.json({
      success: true,
      message: 'Task updated successfully',
      data: task
    });
  } catch (error) {
    if (error.name === 'CastError') {
      return res.status(400).json({ success: false, message: 'Invalid task ID format'
    });
    }
    next(error);
  }
});

// -----
// PATCH /api/tasks/:id - Partial update of a task (user-isolated)
// -----
router.patch('/:id', async (req, res, next) => {
  try {
    const allowedFields = ['title', 'description', 'status', 'priority', 'category', 'dueDate', 'assignedTo', 'tags'];
    const updates = {};
    allowedFields.forEach(field => {
      if (req.body[field] !== undefined) updates[field] = req.body[field];
    });
  }

```

```

    if (Object.keys(updates).length === 0) {
      return res.status(400).json({ success: false, message: 'No valid fields provided f
or update' });
    }

    const task = await Task.findOneAndUpdate(
      { _id: req.params.id, userId: req.user.id },
      { $set: updates },
      { new: true, runValidators: true }
    );

    if (!task) {
      return res.status(404).json({ success: false, message: 'Task not found' });
    }

    res.json({
      success: true,
      message: 'Task patched successfully',
      data: task
    });
  } catch (error) {
    if (error.name === 'CastError') {
      return res.status(400).json({ success: false, message: 'Invalid task ID format'
    });
    }
    next(error);
  }
});

// -----
// DELETE /api/tasks/:id - Delete a task (user-isolated)
// -----
router.delete('/:id', async (req, res, next) => {
  try {
    const task = await Task.findOneAndDelete({ _id: req.params.id, userId: req.user.id
  });
    if (!task) {
      return res.status(404).json({ success: false, message: 'Task not found' });
    }
    res.json({
      success: true,
      message: 'Task deleted successfully',
      data: { deletedId: req.params.id }
    });
  } catch (error) {
    if (error.name === 'CastError') {
      return res.status(400).json({ success: false, message: 'Invalid task ID format'
    });
  }
  next(error);
});

```

```
    }  
  });  
  
  // -----  
  // DELETE /api/tasks - Delete all tasks for the logged in user  
  // -----  
  router.delete('/', async (req, res, next) => {  
    try {  
      const result = await Task.deleteMany({ userId: req.user.id });  
      res.json({  
        success: true,  
        message: `${result.deletedCount} tasks deleted successfully`  
      });  
    } catch (error) {  
      next(error);  
    }  
  });  
  
  module.exports = router;
```

```
import React, { useState, useEffect, useCallback } from 'react';
import { ToastContainer, toast } from 'react-toastify';
import 'react-toastify/dist/ReactToastify.css';
import { taskAPI } from './services/api';
import Navbar from './components/Navbar';
import Dashboard from './components/Dashboard';
import TaskForm from './components/TaskForm';
import TaskList from './components/TaskList';
import HabitTracker from './components/HabitTracker';
import LoginPage from './components/LoginPage';
import './App.css';

export default function App() {
  // — Auth State —————
  const [user, setUser] = useState(null);
  const [authChecked, setAuthChecked] = useState(false);
  const [currentPath, setCurrentPath] = useState(window.location.pathname);

  useEffect(() => {
    const handlePopState = () => setCurrentPath(window.location.pathname);
    window.addEventListener('popstate', handlePopState);
    return () => window.removeEventListener('popstate', handlePopState);
  }, []);

  useEffect(() => {
    const token = localStorage.getItem('todo_token');
    const savedUser = localStorage.getItem('todo_user');
    if (token && savedUser) {
      try {
        setUser(JSON.parse(savedUser));
      } catch {
        localStorage.removeItem('todo_token');
        localStorage.removeItem('todo_user');
        setUser(null);
      }
    } else {
      localStorage.removeItem('todo_token');
      localStorage.removeItem('todo_user');
      setUser(null);
    }
    setAuthChecked(true);
  }, []);

  const handleLogin = (userData) => {
    setUser(userData);
    if (window.location.pathname === '/login') {
      window.history.pushState({}, '', '/');
      setCurrentPath('/');
    }
  };
}
```

```

    }
    toast.success(`👋 Welcome, ${userData.name}!`);
  };

const handleLogout = () => {
  localStorage.removeItem('todo_token');
  localStorage.removeItem('todo_user');
  setUser(null);
  window.history.pushState({}, '', '/login');
  setCurrentPath('/login');
  toast.info('Logged out successfully');
};

const [activeTab, setActiveTab] = useState('tracker');

// ——— Task Board State ———
const [tasks, setTasks] = useState([]);
const [stats, setStats] = useState(null);
const [loading, setLoading] = useState(true);
const [showForm, setShowForm] = useState(false);
const [editingTask, setEditingTask] = useState(null);
const [activeView] = useState('board');
const [filters, setFilters] = useState({ status: '', priority: '', search: '', sortBy:
'createdAt', order: 'desc' });
const [pagination, setPagination] = useState({ currentPage: 1, totalPages: 1, totalCou
nt: 0, limit: 20 });

const fetchTasks = useCallback(async () => {
  try {
    setLoading(true);
    const params = { ...filters, page: pagination.currentPage, limit: 20 };
    Object.keys(params).forEach(k => !params[k] && delete params[k]);
    const res = await taskAPI.getAll(params);
    setTasks(res.data.data);
    setPagination(prev => ({ ...prev, ...res.data.pagination }));
  } catch (err) { toast.error(`Failed to fetch tasks: ${err.message}`); }
  finally { setLoading(false); }
}, [filters, pagination.currentPage]);

const fetchStats = useCallback(async () => {
  try { const res = await taskAPI.getStats(); setStats(res.data.data); }
  catch (err) { console.error(err.message); }
}, []);

useEffect(() => {
  if (activeTab === 'tasks') { fetchTasks(); fetchStats(); }
}, [activeTab, fetchTasks, fetchStats]);

const handleCreate = async (formData) => {
  try {
    await taskAPI.create(formData);
  }

```

```

        toast.success('✅ Task created!');
        setShowForm(false);
        fetchTasks(); fetchStats();
    } catch (err) { toast.error(`❌ ${err.message}`); throw err; }
};

const handleUpdate = async (id, formData) => {
    try {
        await taskAPI.update(id, formData);
        toast.success('✅ Task updated!');
        setEditingTask(null); setShowForm(false);
        fetchTasks(); fetchStats();
    } catch (err) { toast.error(`❌ ${err.message}`); throw err; }
};

const handleStatusChange = async (id, newStatus) => {
    try {
        await taskAPI.patch(id, { status: newStatus });
        toast.success(`Status → "${newStatus}"`);
        fetchTasks(); fetchStats();
    } catch (err) { toast.error(`❌ ${err.message}`); }
};

const handleDelete = async (id) => {
    if (!window.confirm('Delete this task?')) return;
    try {
        await taskAPI.delete(id);
        toast.success('🗑️ Task deleted!');
        fetchTasks(); fetchStats();
    } catch (err) { toast.error(`❌ ${err.message}`); }
};

const handleFilterChange = (newFilters) => {
    setFilters(prev => ({ ...prev, ...newFilters }));
    setPagination(prev => ({ ...prev, currentPage: 1 }));
};

// — Auth Gate & Route Check —————
if (!authChecked) return null; // wait for localStorage check
if (!user || currentPath === '/login') return (
    <>
        <LoginPage onLogin={handleLogin} />
        <ToastContainer position="bottom-right" autoClose={2500} theme="dark" />
    </>
);

return (
    <div className="app">
        <Navbar
            activeTab={activeTab}
            setActiveTab={setActiveTab}

```

```

        onNewTask={() => { setEditingTask(null); setShowForm(true); }}
        user={user}
        onLogout={handleLogout}
    />

<main className="app__main">
    {/* — Habit Tracker Tab — */}
    {activeTab === 'tracker' && <HabitTracker />}

    {/* — Task Board Tab — */}
    {activeTab === 'tasks' && (
        <>
            <Dashboard stats={stats} />
            {showForm && (
                <TaskForm
                    task={editingTask}
                    onSubmit={editingTask
                        ? (data) => handleUpdate(editingTask._id, data)
                        : handleCreate}
                    onClose={() => { setShowForm(false); setEditingTask(null); }}
                />
            )}
            <TaskList
                tasks={tasks}
                loading={loading}
                filters={filters}
                onFilterChange={handleFilterChange}
                onEdit={(t) => { setEditingTask(t); setShowForm(true); }}
                onDelete={handleDelete}
                onStatusChange={handleStatusChange}
                onNewTask={() => { setEditingTask(null); setShowForm(true); }}
                pagination={pagination}
                onPageChange={(page) => setPagination(prev => ({ ...prev, currentPage: pag
e })))}
                activeView={activeView}
            />
        </>
    )}
</main>

<ToastContainer
    position="bottom-right"
    autoClose={2500}
    theme="dark"
    newestOnTop
    closeOnClick
    pauseOnHover
    />
</div>

```

```
);  
}
```

```

import React from 'react';

export default function Navbar({ activeTab, setActiveTab, onNewTask, user, onLogout }) {
  return (
    <nav className="navbar">
      <div className="navbar__inner">
        {/* Brand */}
        <div className="navbar__brand">
          <div className="navbar__brand-icon"><img alt="Habit Tracker icon" data-bbox="215 245 235 260"/></div>
          <span className="navbar__brand-text">TO-DO-TRACKER</span>
        </div>

        {/* Tab Navigation */}
        <div className="navbar__tabs">
          <button
            className={`navbar__tab ${activeTab === 'tracker' ? 'active' : ''}`}
            onClick={() => setActiveTab('tracker')}
          >
            <img alt="Habit Tracker icon" data-bbox="215 425 235 440"/> Habit Tracker
          </button>
          <button
            className={`navbar__tab ${activeTab === 'tasks' ? 'active' : ''}`}
            onClick={() => setActiveTab('tasks')}
          >
            <img alt="Task Board icon" data-bbox="215 535 235 550"/> Task Board
          </button>
        </div>

        {/* Actions & User Profile */}
        <div className="navbar__actions">
          {activeTab === 'tasks' && (
            <button className="btn btn-primary" onClick={onNewTask}>
              + New Task
            </button>
          )}

          {user && (
            <div className="navbar__user">
              <span className="navbar__user-avatar">
                {user.name ? user.name[0].toUpperCase() : 'U'}
              </span>
              <span className="navbar__user-name">{user.name}</span>
              <button
                className="btn btn-outline navbar__logout-btn"
                onClick={onLogout}
                title="Sign Out"
              >
                <img alt="Sign Out icon" data-bbox="255 945 275 960"/> Sign Out
              </button>
            </div>
          )}
        </div>
      </div>
    </nav>
  );
}

```

```
        </button>
      </div>
    )}
  </div>
</div>
</nav>
);
}
```

```
import React, { useState } from 'react';
import { authAPI } from '../services/authAPI';

export default function LoginPage({ onLogin }) {
  const [mode, setMode] = useState('login'); // 'login' | 'register'
  const [form, setForm] = useState({ name: '', email: '', password: '' });
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  const set = (field) => (e) => {
    setForm(f => ({ ...f, [field]: e.target.value }));
    setError('');
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!form.email || !form.password) { setError('Email and password are required'); return; }
    if (mode === 'register' && !form.name) { setError('Name is required'); return; }
    if (form.password.length < 6) { setError('Password must be at least 6 characters'); return; }

    setLoading(true);
    try {
      const res = mode === 'login'
        ? await authAPI.login({ email: form.email, password: form.password })
        : await authAPI.register({ name: form.name, email: form.email, password: form.password });

      const { token, user } = res.data;
      localStorage.setItem('todo_token', token);
      localStorage.setItem('todo_user', JSON.stringify(user));
      onLogin(user, token);
    } catch (err) {
      setError(err.response?.data?.message || err.message || 'Something went wrong');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="auth-page">
      {/* Background decoration */}
      <div className="auth-bg">
        <div className="auth-blob auth-blob-1" />
        <div className="auth-blob auth-blob-2" />
      </div>
    </div>
  );
}
```

```

<div className="auth-card">
  {/* Logo */}
  <div className="auth-logo">
    <div className="auth-logo-icon">☑</div>
    <h1 className="auth-logo-text">TO-DO-TRACKER</h1>
  </div>

  <p className="auth-tagline">Track habits. Hit goals. Build streaks.</p>

  {/* Tab Toggle */}
  <div className="auth-tabs">
    <button className={`auth-tab ${mode === 'login' ? 'active' : ''}`}
      onClick={() => { setMode('login'); setError(''); }}>
      Sign In
    </button>
    <button className={`auth-tab ${mode === 'register' ? 'active' : ''}`}
      onClick={() => { setMode('register'); setError(''); }}>
      Create Account
    </button>
  </div>

  {/* Form */}
  <form className="auth-form" onSubmit={handleSubmit}>
    {mode === 'register' && (
      <div className="auth-field">
        <label className="auth-label">Full Name</label>
        <input className="auth-input" type="text" placeholder="Aniket Mohanty"
          value={form.name} onChange={set('name')} autoFocus />
      </div>
    )}

    <div className="auth-field">
      <label className="auth-label">Email Address</label>
      <input className="auth-input" type="email" placeholder="you@example.com"
        value={form.email} onChange={set('email')} autoFocus={mode === 'login'} />
    </div>

    <div className="auth-field">
      <label className="auth-label">Password</label>
      <input className="auth-input" type="password" placeholder="....."
        value={form.password} onChange={set('password')} />
      {mode === 'login' && (
        <span className="auth-hint">Minimum 6 characters</span>
      )}
    </div>

    {error && (
      <div className="auth-error">⚠ {error}</div>
    )}

    <button className="auth-submit" type="submit" disabled={loading}>

```

```

        {loading
          ? <><span className="auth-spinner" /> {mode === 'login' ? 'Signing in...'
: 'Creating account...'}</>
          : mode === 'login' ? '🚀 Sign In' : '🌟 Create Account'
        }
      </button>
    </form>

    {/* Switch mode */}
    <p className="auth-switch">
      {mode === 'login'
        ? <>Don't have an account? <button className="auth-link" onClick={() => { se
tMode('register'); setError(''); }}>Create one free</button></>
        : <>Already have an account? <button className="auth-link" onClick={() => {
setMode('login'); setError(''); }}>Sign in</button></>
      }
    </p>

    {/* Features */}
    <div className="auth-features">
      <span>📊 Progress Charts</span>
      <span>📌 Habit Tracking</span>
      <span>☁️ Cloud Sync</span>
    </div>
  </div>
</div>
);
}

```

```

import React, { useState, useEffect, useCallback } from 'react';
import { habitAPI, completionAPI } from '../services/habitAPI';
import { toast } from 'react-toastify';
import HabitTaskForm from './HabitTaskForm';
import ProgressChart from './ProgressChart';

const DAY_ABBR = ['Su', 'Mo', 'Tu', 'We', 'Th', 'Fr', 'Sa'];
const MONTH_NAMES = ['January', 'February', 'March', 'April', 'May', 'June',
  'July', 'August', 'September', 'October', 'November', 'December'];

function getMonthDays(year, month) {
  const days = [];
  const d = new Date(year, month, 1);
  while (d.getMonth() === month) { days.push(new Date(d)); d.setDate(d.getDate() + 1); }
  return days;
}

function toDateStr(d) { return d.toISOString().split('T')[0]; }

function isScheduled(task, date) {
  if (task.type === 'fixed') return true;
  return task.scheduledDays.includes(date.getDay());
}

export default function HabitTracker() {
  const today = new Date();
  const [year, setYear] = useState(today.getFullYear());
  const [month, setMonth] = useState(today.getMonth());
  const [tasks, setTasks] = useState([]);
  const [completions, setCompletions] = useState({}); // {taskId_date: true/false}
  const [loading, setLoading] = useState(true);
  const [showForm, setShowForm] = useState(false);
  const [editTask, setEditTask] = useState(null);
  const [chartKey, setChartKey] = useState(0); // force chart re-fetch

  const monthStr = `${year}-${String(month + 1).padStart(2, '0')}`;
  const days = getMonthDays(year, month);

  // — Fetch tasks + completions —————
  const fetchData = useCallback(async () => {
    setLoading(true);
    try {
      const [tasksRes, compRes] = await Promise.all([
        habitAPI.getAll(),
        completionAPI.getByMonth(monthStr)
      ]);
      setTasks(tasksRes.data.data);
      // Build lookup map: "taskId_date" -> completed
    }
  });

```

```

    const map = {};
    compRes.data.data.forEach(c => {
      map[`${c.taskId}_${c.date}`] = c.completed;
    });
    setCompletions(map);
  } catch (e) { toast.error(e.message); }
  finally { setLoading(false); }
}, [monthStr]);

useEffect(() => { fetchData(); }, [fetchData]);

// — Toggle completion —————
const toggleCell = async (taskId, dateStr) => {
  const key = `${taskId}_${dateStr}`;
  const prev = completions[key];
  // Optimistic update
  setCompletions(c => ({ ...c, [key]: !prev }));
  try {
    await completionAPI.toggle(taskId, dateStr);
    setChartKey(k => k + 1); // refresh chart
  } catch (e) {
    setCompletions(c => ({ ...c, [key]: prev }));
    toast.error(e.message);
  }
};

// — CRUD handlers —————
const handleCreate = async (data) => {
  try {
    await habitAPI.create(data);
    toast.success('✅ Habit task added!');
    setShowForm(false);
    fetchData();
  } catch (e) { toast.error(e.message); }
};

const handleUpdate = async (data) => {
  try {
    await habitAPI.update(editTask._id, data);
    toast.success('✅ Habit task updated!');
    setEditTask(null);
    fetchData();
  } catch (e) { toast.error(e.message); }
};

const handleDelete = async (id) => {
  if (!window.confirm('Delete this habit task?')) return;
  try {
    await habitAPI.delete(id);
    toast.success('🗑 Task deleted!');
    fetchData();
  }
};

```

```

    } catch (e) { toast.error(e.message); }
  };

  // — Month nav —————
  const prevMonth = () => { if (month === 0) { setYear(y => y - 1); setMonth(11); } else
setMonth(m => m - 1); };
  const nextMonth = () => { if (month === 11) { setYear(y => y + 1); setMonth(0); } else
setMonth(m => m + 1); };

  // — Per-task row completion % —————
  function taskPct(task) {
    const scheduled = days.filter(d => isScheduled(task, d));
    if (!scheduled.length) return 0;
    const done = scheduled.filter(d => completions[`_${task._id}_${toDateStr(d)}`]).length
h;
    return Math.round((done / scheduled.length) * 100);
  }

  // — Per-day column completion % —————
  function dayPct(day) {
    const scheduled = tasks.filter(t => isScheduled(t, day));
    if (!scheduled.length) return null;
    const done = scheduled.filter(t => completions[`_${t._id}_${toDateStr(day)}`]).length
h;
    return Math.round((done / scheduled.length) * 100);
  }

  const isToday = (d) => toDateStr(d) === toDateStr(today);
  const isFuture = (d) => d > today;

  return (
    <div className="habit-tracker">
      {/* Progress Chart */}
      <ProgressChart key={chartKey} />

      {/* Header */}
      <div className="ht-header">
        <div className="ht-month-nav">
          <button className="ht-nav-btn" onClick={prevMonth}><</button>
          <h2 className="ht-month-title">{MONTH_NAMES[month]} {year}</h2>
          <button className="ht-nav-btn" onClick={nextMonth}>>>/button>
        </div>
        <button className="btn btn-primary btn-sm" onClick={() => { setEditTask(null); s
etShowForm(true); }}>
          + Add Habit
        </button>
      </div>

      {/* Task Form Modal */}
      {(showForm || editTask) && (
        <HabitTaskForm

```



```

<td key={i} className={`ht-td-daypct ${isToday(d) ? 'ht-today-col' :
''}`}>
    {pct !== null && !isFuture(d) ? (
      <span className="ht-daypct-val" style={{
        color: pct === 100 ? 'var(--success)' : pct >= 50 ? 'var(--war
ning)' : 'var(--danger)'}
      >{pct}%</span>
    ) : <span style={{ color: 'var(--text-dim)' }}></span>}
  </td>
);
}}
</tr>
</thead>
<tbody>
  {tasks.map((task, rowIdx) => {
    const pct = taskPct(task);
    return (
      <tr key={task._id} className="ht-row">
        {/* Task name */}
        <td className="ht-td-label">
          <div className="ht-task-info">
            <span className="ht-task-dot" style={{ background: task.color }}
/>
            <span className="ht-task-name" style={{ color: task.color }}>{ta
sk.name}</span>
            <span className={`ht-type-badge ${task.type}`}>
              {task.type === 'fixed' ? '📌' : '📅'}
            </span>
          </div>
          <div className="ht-task-actions">
            <button className="ht-action-btn" onClick={() => setEditTask(tas
k)} title="Edit">✏️</button>
            <button className="ht-action-btn" onClick={() => handleDelete(ta
sk._id)} title="Delete">🗑️</button>
          </div>
        </td>
        {/* Row % */}
        <td className="ht-td-pct">
          <div className="ht-pct-wrap">
            <span className="ht-pct-val" style={{
              color: pct === 100 ? 'var(--success)' : pct >= 50 ? 'var(--war
ning)' : 'var(--text-muted)'}
            >{pct}%</span>
            <div className="ht-pct-bar">
              <div className="ht-pct-fill" style={{ width: `${pct}%`, backgr
ound: task.color }} />
            </div>
          </div>
        </td>
        {/* Checkboxes */}
        {days.map((d, ci) => {

```

```

const dateStr = toDateStr(d);
const scheduled = isScheduled(task, d);
const done = !!completions[`${task._id}_${dateStr}`];
const future = isFuture(d) && !isToday(d);

// Color cycling by week
const weekIdx = Math.floor((d.getDate() - 1) / 7);
const weekColors = ['#818cf8', '#f472b6', '#2dd4bf', '#fbbf24', '#f973
16'];

const cellColor = scheduled ? (task.color || weekColors[weekIdx %
weekColors.length]) : 'transparent';

return (
  <td key={ci} className={`ht-td-cell ${isToday(d) ? 'ht-today-co
1' : ''}`>

    {scheduled ? (
      <button
        className={`ht-checkbox ${done ? 'checked' : ''} ${future
? 'future' : ''}`}
        style={done ? { borderColor: cellColor, background: cellCo
lor + '33' } : { borderColor: cellColor + '66' }}
        onClick={() => !future && toggleCell(task._id, dateStr)}
        disabled={future}
        title={` ${task.name} - ${dateStr}`}
      >
        {done && <span className="ht-check" style={{ color: cellCo
lor }}>✓</span>}
      </button>
    ) : (
      <div className="ht-na" title="Not scheduled">•</div>
    )}
  </td>
);
}}}
</tr>

);
}}}
</tbody>
</table>
</div>
)}

{/* Legend */}
<div className="ht-legend">
  <span className="ht-legend-item"><span className="ht-legend-dot" style={{ backgr
ound: '#818cf8' }} /> Fixed task (every day)</span>
  <span className="ht-legend-item"><span className="ht-legend-dot" style={{ backgr
ound: '#fbbf24' }} /> Variable task (selected days)</span>
  <span className="ht-legend-item"><span style={{ opacity: 0.4 }}>•</span> Not sch
eduled</span>
  <span className="ht-legend-item"><span style={{ color: 'var(--success)' }}>✓</sp

```

```
an> Completed</span>
      </div>
    </div>
  );
}
```

```

import React, { useState } from 'react';

const DAY_NAMES = ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'];
const COLORS = ['#818cf8', '#60a5fa', '#34d399', '#f472b6', '#fbbf24', '#f97316', '#2dd4bf', '#a78bfa', '#fb923c', '#4ade80'];

export default function HabitTaskForm({ task, onSubmit, onClose }) {
  const [form, setForm] = useState({
    name: task?.name || '',
    type: task?.type || 'fixed',
    scheduledDays: task?.scheduledDays || [],
    color: task?.color || COLORS[0],
  });
  const [submitting, setSubmitting] = useState(false);
  const [err, setErr] = useState('');

  const toggleDay = (day) => {
    setForm(f => ({
      ...f,
      scheduledDays: f.scheduledDays.includes(day)
        ? f.scheduledDays.filter(d => d !== day)
        : [...f.scheduledDays, day]
    }));
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!form.name.trim()) { setErr('Name is required'); return; }
    if (form.type === 'variable' && form.scheduledDays.length === 0) {
      setErr('Select at least one day for variable task'); return;
    }
    setSubmitting(true);
    try {
      await onSubmit(form);
    } finally { setSubmitting(false); }
  };

  return (
    <div className="modal-overlay" onClick={e => e.target === e.currentTarget && onClose}
    >>
    <div className="modal" style={{ maxWidth: 460 }}>
      <div className="modal__header">
        <h2 className="modal__title">{task ? '✏ Edit Habit' : '+ New Habit Task'}</
h2>
        <button className="modal__close" onClick={onClose}>X</button>
      </div>
      <div className="modal__body">

```

```

<form onSubmit={handleSubmit}>
  {/* Name */}
  <div className="form-group">
    <label className="form-label">Task Name <span>*</span></label>
    <input className="form-control" type="text" placeholder="e.g. Exercise for
30 minutes"
      value={form.name} onChange={e => { setForm(f => ({ ...f, name: e.target.
value })); setErr(''); }}
      maxLength={80} autoFocus />
    </div>

    {/* Type */}
    <div className="form-group">
      <label className="form-label">Task Type</label>
      <div className="type-toggle">
        <button type="button"
          className={`type-btn ${form.type === 'fixed' ? 'active' : ''}`}
          onClick={() => setForm(f => ({ ...f, type: 'fixed', scheduledDays: []
})))}>
          📌 Fixed <small>Every day</small>
        </button>
        <button type="button"
          className={`type-btn ${form.type === 'variable' ? 'active' : ''}`}
          onClick={() => setForm(f => ({ ...f, type: 'variable' })))}>
          📅 Variable <small>Specific days</small>
        </button>
      </div>

      {/* Day Selector for Variable */}
      {form.type === 'variable' && (
        <div className="form-group">
          <label className="form-label">Scheduled Days</label>
          <div className="day-selector">
            {DAY_NAMES.map((name, i) => (
              <button type="button" key={i}
                className={`day-btn ${form.scheduledDays.includes(i) ? 'active' :
''}`}
                onClick={() => toggleDay(i)}>
                {name}
              </button>
            ))}
          </div>
        </div>
      )}

      {/* Color */}
      <div className="form-group">
        <label className="form-label">Color</label>
        <div className="color-picker">
          {COLORS.map(c => (

```

```

        <button type="button" key={c}
          className={`color-dot ${form.color === c ? 'active' : ''}`}
          style={{ background: c }}
          onClick={() => setForm(f => ({ ...f, color: c }))}
        />
      )}}
    </div>
  </div>

  {err && <p style={{ color: 'var(--danger)', fontSize: '0.82rem', marginBotto
m: 12 }}>{err}</p>}

  <div className="form-actions">
    <button type="button" className="btn btn-secondary" onClick={onClose}>Canc
el</button>
    <button type="submit" className="btn btn-primary" disabled={submitting}>
      {submitting ? '⌚ Saving...' : task ? '💾 Update' : '+ Add Task'}
    </button>
  </div>
</form>
</div>
</div>
</div>
);
}

```

```

import React, { useEffect, useState, useCallback } from 'react';
import { completionAPI } from '../services/habitAPI';

// — Smooth bezier path generator —————
function smoothPath(points, W, H, pad = 20) {
  if (points.length < 2) return { line: '', area: '' };
  const xs = points.map((_, i) => pad + (i / (points.length - 1)) * (W - 2 * pad));
  const ys = points.map(p => H - pad - (p / 100) * (H - 2 * pad));

  let line = `M ${xs[0]} ${ys[0]}`;
  let area = `M ${xs[0]} ${H - pad} L ${xs[0]} ${ys[0]}`;

  for (let i = 1; i < points.length; i++) {
    const cpx = (xs[i - 1] + xs[i]) / 2;
    const seg = `C ${cpx} ${ys[i - 1]} ${cpx} ${ys[i]} ${xs[i]} ${ys[i]}`;
    line += seg;
    area += seg;
  }
  area += ` L ${xs[xs.length - 1]} ${H - pad} Z`;
  return { line, area };
}

// — X-axis label (every 5 days) —————
function formatLabel(dateStr) {
  const d = new Date(dateStr);
  return `${d.getDate()}/${d.getMonth() + 1}`;
}

const W = 900, H = 180, PAD = 28;

export default function ProgressChart() {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(true);
  const [hovered, setHovered] = useState(null);

  const load = useCallback(async () => {
    try {
      setLoading(true);
      const res = await completionAPI.getChart(30);
      setData(res.data.data);
    } catch (e) {
      console.error('Chart load error:', e.message);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => { load(); }, [load]);

  if (loading) return (

```

```

    <div className="chart-wrap">
      <div className="chart-loading">
        <div className="spinner" style={{ width: 28, height: 28, borderWidth: 2 }} />
        <span>Loading chart...</span>
      </div>
    </div>
  );

  const pcts = data.map(d => d.pct);
  const { line, area } = smoothPath(pcts, W, H, PAD);
  const avgPct = pcts.length ? Math.round(pcts.reduce((a, b) => a + b, 0) / pcts.length)
: 0;

  // X positions for labels and hover
  const xs = pcts.map((_, i) => PAD + (i / (pcts.length - 1 || 1)) * (W - 2 * PAD));
  const ys = pcts.map(p => H - PAD - (p / 100) * (H - 2 * PAD));

  return (
    <div className="chart-wrap">
      <div className="chart-header">
        <div>
          <h2 className="chart-title">Daily Completion Rate</h2>
          <p className="chart-sub">Last 30 days – Avg: <strong style={{ color: 'var(--primary)' }}>{avgPct}%</strong></p>
        </div>
        <div className="chart-badges">
          {[0, 25, 50, 75, 100].map(v => (
            <span key={v} className="chart-badge">{v}%</span>
          ))}
        </div>
      </div>

      <div className="chart-svg-wrap">
        <svg
          viewBox={`0 0 ${W} ${H}`}
          preserveAspectRatio="none"
          className="chart-svg"
          onMouseLeave={() => setHovered(null)}
        >
          <defs>
            <linearGradient id="areaGrad" x1="0" y1="0" x2="0" y2="1">
              <stop offset="0%" stopColor="#818cf8" stopOpacity="0.35" />
              <stop offset="100%" stopColor="#818cf8" stopOpacity="0.02" />
            </linearGradient>
            {/* Horizontal gridlines */}
            {[25, 50, 75].map(v => {
              const y = H - PAD - (v / 100) * (H - 2 * PAD);
              return (
                <line key={v} x1={PAD} y1={y} x2={W - PAD} y2={y}
                  stroke="#334155" strokeWidth="1" strokeDasharray="4,4" />
              );
            )}
          </defs>

```

```

    }}}
  </defs>

  {/* Gridlines */}
  {[25, 50, 75].map(v => {
    const y = H - PAD - (v / 100) * (H - 2 * PAD);
    return <line key={v} x1={PAD} y1={y} x2={W - PAD} y2={y}
      stroke="#334155" strokeWidth="1" strokeDasharray="4,4" />;
  })}

  {/* Area fill */}
  {pcts.length > 1 && <path d={area} fill="url(#areaGrad)" />}

  {/* Line */}
  {pcts.length > 1 && (
    <path d={line} fill="none" stroke="#818cf8" strokeWidth="2.5"
      strokeLinecap="round" strokeLinejoin="round" />
  )}

  {/* Hover dots + invisible hit areas */}
  {data.map((d, i) => (
    <g key={i}>
      <rect
        x={xs[i] - 10} y={0} width={20} height={H}
        fill="transparent"
        onMouseEnter={() => setHovered(i)}
      />
      {hovered === i && (
        <>
          <line x1={xs[i]} y1={PAD} x2={xs[i]} y2={H - PAD}
            stroke="#475569" strokeWidth="1" strokeDasharray="3,3" />
          <circle cx={xs[i]} cy={ys[i]} r={5} fill="#818cf8" stroke="#1e293b" st
rokeWidth={2} />
          <rect x={xs[i] - 36} y={ys[i] - 28} width={72} height={22} rx={5}
            fill="#334155" stroke="#475569" strokeWidth={1} />
          <text x={xs[i]} y={ys[i] - 13} textAnchor="middle"
            fill="#f1f5f9" fontSize={10} fontWeight="600">
            {d.pct}% - {formatLabel(d.date)}
          </text>
        </>
      )}
    </g>
  )})}

  {/* X-axis labels every 5 days */}
  {data.map((d, i) => {
    if (i % 5 !== 0 && i !== data.length - 1) return null;
    return (
      <text key={i} x={xs[i]} y={H - 4} textAnchor="middle"
        fill="#64748b" fontSize={9}>
        {formatLabel(d.date)}
      </text>
    )
  })}

```

```

        </text>
    );
  })}

  { /* Y-axis labels */
  [0, 50, 100].map(v => {
    const y = H - PAD - (v / 100) * (H - 2 * PAD);
    return (
      <text key={v} x={PAD - 6} y={y + 4} textAnchor="end"
        fill="#64748b" fontSize={9}>
        {v}%
      </text>
    );
  })}
</svg>
</div>
</div>
);
}

```

```
import React from 'react';

const statItems = [
  {
    key: 'total',
    label: 'Total Tasks',
    icon: '📋',
    bg: '#ede9fe',
    getValue: (s) => s?.total ?? 0,
  },
  {
    key: 'todo',
    label: 'To Do',
    icon: '🟦',
    bg: '#f1f5f9',
    getValue: (s) => s?.byStatus?.todo ?? 0,
  },
  {
    key: 'inprogress',
    label: 'In Progress',
    icon: '🟡',
    bg: '#e0f2fe',
    getValue: (s) => s?.byStatus?.['in-progress'] ?? 0,
  },
  {
    key: 'completed',
    label: 'Completed',
    icon: '🟢',
    bg: '#dcfce7',
    getValue: (s) => s?.byStatus?.completed ?? 0,
  },
  {
    key: 'high',
    label: 'High Priority',
    icon: '🔴',
    bg: '#fef2f2',
    getValue: (s) => s?.byPriority?.high ?? 0,
  },
  {
    key: 'overdue',
    label: 'Overdue',
    icon: '⚠️',
    bg: '#fff7ed',
    getValue: (s) => s?.overdue ?? 0,
  },
];

function Dashboard({ stats }) {
```

```

const completionRate = stats?.completionRate ?? '0.0';

return (
  <div className="dashboard">
    <div className="dashboard__grid">
      {statItems.map((item) => (
        <div className="stat-card" key={item.key}>
          <div className="stat-card__icon" style={{ background: item.bg }}>
            {item.icon}
          </div>
          <div className="stat-card__info">
            <div className="stat-card__value">{item.getValue(stats)}</div>
            <div className="stat-card__label">{item.label}</div>
          </div>
        </div>
      ))}
      <div className="stat-card">
        <div className="stat-card__icon" style={{ background: '#fdf4ff' }}>

        </div>
        <div className="stat-card__info">
          <div className="stat-card__value">{completionRate}%</div>
          <div className="stat-card__label">Completion Rate</div>
        </div>
      </div>
    </div>
  </div>
);
}

export default Dashboard;

```

```
import React, { useState, useEffect } from 'react';

const INITIAL_FORM = {
  title: '',
  description: '',
  status: 'todo',
  priority: 'medium',
  category: 'General',
  dueDate: '',
  assignedTo: '',
  tags: '',
};

function TaskForm({ task, onSubmit, onClose }) {
  const [form, setForm] = useState(INITIAL_FORM);
  const [submitting, setSubmitting] = useState(false);
  const [errors, setErrors] = useState({});

  // Populate form when editing
  useEffect(() => {
    if (task) {
      setForm({
        title: task.title || '',
        description: task.description || '',
        status: task.status || 'todo',
        priority: task.priority || 'medium',
        category: task.category || 'General',
        dueDate: task.dueDate ? task.dueDate.split('T')[0] : '',
        assignedTo: task.assignedTo || '',
        tags: Array.isArray(task.tags) ? task.tags.join(', ') : '',
      });
    }
  }, [task]);

  const validate = () => {
    const errs = {};
    if (!form.title.trim()) errs.title = 'Title is required';
    else if (form.title.trim().length < 3) errs.title = 'Title must be at least 3 characters';
    if (form.description.length > 500) errs.description = 'Max 500 characters';
    return errs;
  };

  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm(prev => ({ ...prev, [name]: value }));
    if (errors[name]) setErrors(prev => ({ ...prev, [name]: '' }));
  };
}
```

```

const handleSubmit = async (e) => {
  e.preventDefault();
  const errs = validate();
  if (Object.keys(errs).length > 0) {
    setErrors(errs);
    return;
  }

  setSubmitting(true);
  try {
    const payload = {
      ...form,
      title: form.title.trim(),
      description: form.description.trim(),
      tags: form.tags
        ? form.tags.split(',').map(t => t.trim()).filter(Boolean)
        : [],
      dueDate: form.dueDate || null,
    };
    await onSubmit(payload);
  } catch (err) {
    // Error is already handled by App.js toast
  } finally {
    setSubmitting(false);
  }
};

return (
  <div className="modal-overlay" onClick={(e) => e.target === e.currentTarget && onClose()}>
    <div className="modal">
      <div className="modal__header">
        <h2 className="modal__title">
          {task ? '✏ Edit Task' : '+ Create New Task'}
        </h2>
        <button className="modal__close" onClick={onClose} title="Close">X</button>
      </div>

      <div className="modal__body">
        <form onSubmit={handleSubmit} noValidate>
          {/* Title */}
          <div className="form-group">
            <label className="form-label">Title <span>*</span></label>
            <input
              type="text"
              name="title"
              className={`form-control ${errors.title ? 'error' : ''}`}
              placeholder="e.g., Design homepage wireframes"
              value={form.title}
              onChange={handleChange}
            />
          </div>
        </form>
      </div>
    </div>
  </div>
);

```

```

        maxLength={100}
        autoFocus
      />
      {errors.title && <span style={{ color: 'var(--danger)', fontSize: '0.78rem',
marginTop: '4px', display: 'block' }}>{errors.title}</span>}
    </div>

    {/* Description */}
    <div className="form-group">
      <label className="form-label">Description</label>
      <textarea
        name="description"
        className="form-control"
        placeholder="Add details about this task..."
        value={form.description}
        onChange={handleChange}
        rows={3}
        maxLength={500}
      />
      <span style={{ fontSize: '0.72rem', color: 'var(--text-muted)', float: 'right' }}>
        {form.description.length}/500
      </span>
    </div>

    {/* Status & Priority */}
    <div className="form-row">
      <div className="form-group">
        <label className="form-label">Status</label>
        <select name="status" className="form-control" value={form.status} onChange={handleChange}>
          <option value="todo">📌 To Do</option>
          <option value="in-progress">🔄 In Progress</option>
          <option value="completed">✅ Completed</option>
        </select>
      </div>
      <div className="form-group">
        <label className="form-label">Priority</label>
        <select name="priority" className="form-control" value={form.priority} onChange={handleChange}>
          <option value="low">🟢 Low</option>
          <option value="medium">🟡 Medium</option>
          <option value="high">🔴 High</option>
        </select>
      </div>
    </div>

    {/* Category & Due Date */}
    <div className="form-row">
      <div className="form-group">
        <label className="form-label">Category</label>

```

```

        <input
            type="text"
            name="category"
            className="form-control"
            placeholder="e.g., Design, Dev, Marketing"
            value={form.category}
            onChange={handleChange}
            maxLength={50}
        />
    </div>
    <div className="form-group">
        <label className="form-label">Due Date</label>
        <input
            type="date"
            name="dueDate"
            className="form-control"
            value={form.dueDate}
            onChange={handleChange}
        />
    </div>
</div>

{/* Assigned To */}
<div className="form-group">
    <label className="form-label">Assigned To</label>
    <input
        type="text"
        name="assignedTo"
        className="form-control"
        placeholder="e.g., John Doe"
        value={form.assignedTo}
        onChange={handleChange}
        maxLength={50}
    />
</div>

{/* Tags */}
<div className="form-group">
    <label className="form-label">Tags <span style={{ fontWeight: 400, color:
'var(--text-muted)' }}>(comma-separated)</span></label>
    <input
        type="text"
        name="tags"
        className="form-control"
        placeholder="e.g., frontend, urgent, bug"
        value={form.tags}
        onChange={handleChange}
    />
</div>

{/* Actions */}

```

```

    <div className="form-actions">
      <button type="button" className="btn btn-secondary" onClick={onClose}>
        Cancel
      </button>
      <button type="submit" className="btn btn-primary" disabled={submitting}>
        {submitting ? '⌚ Saving...' : task ? '💾 Update Task' : '+ Create Tas
k'}
      </button>
    </div>
  </form>
</div>
</div>
</div>
);
}

export default TaskForm;

```

```

import React, { useState } from 'react';
import TaskCard from './TaskCard';

function formatDate(dateStr) {
  if (!dateStr) return null;
  return new Date(dateStr).toLocaleDateString('en-IN', { day: '2-digit', month: 'short',
year: 'numeric' });
}

function isOverdue(dueDate, status) {
  if (!dueDate || status === 'completed') return false;
  return new Date() > new Date(dueDate);
}

// — Filter Bar —————
function FilterBar({ filters, onFilterChange }) {
  const [search, setSearch] = useState(filters.search || '');

  const handleSearchSubmit = (e) => {
    e.preventDefault();
    onFilterChange({ search });
  };

  return (
    <div className="filter-bar">
      {/* Search */}
      <form className="filter-bar__search" onSubmit={handleSearchSubmit}>
        <span className="filter-bar__search-icon">🔍</span>
        <input
          type="text"
          placeholder="Search tasks..."
          value={search}
          onChange={(e) => {
            setSearch(e.target.value);
            if (!e.target.value) onFilterChange({ search: '' });
          }}
          onKeyDown={(e) => e.key === 'Enter' && handleSearchSubmit(e)}
        />
      </form>

      {/* Status Filter */}
      <select
        value={filters.status}
        onChange={(e) => onFilterChange({ status: e.target.value })}
      >
        <option value="">All Status</option>
        <option value="todo">📌 To Do</option>
        <option value="in-progress">🔄 In Progress</option>
      </select>
    </div>
  );
}

```

```

      <option value="completed">✅ Completed</option>
    </select>

    { /* Priority Filter */ }
    <select
      value={filters.priority}
      onChange={(e) => onFilterChange({ priority: e.target.value })}
    >
      <option value="">All Priority</option>
      <option value="low">🟢 Low</option>
      <option value="medium">🟡 Medium</option>
      <option value="high">🔴 High</option>
    </select>

    { /* Sort */ }
    <select
      value={filters.sortBy}
      onChange={(e) => onFilterChange({ sortBy: e.target.value })}
    >
      <option value="createdAt">Sort: Created</option>
      <option value="dueDate">Sort: Due Date</option>
      <option value="priority">Sort: Priority</option>
      <option value="title">Sort: Title</option>
    </select>

    { /* Order */ }
    <select
      value={filters.order}
      onChange={(e) => onFilterChange({ order: e.target.value })}
    >
      <option value="desc">↓ Desc</option>
      <option value="asc">↑ Asc</option>
    </select>

    { /* Clear Filters */ }
    {(filters.status || filters.priority || filters.search) && (
      <button
        className="btn btn-secondary btn-sm"
        onClick={() => { onFilterChange({ status: '', priority: '', search: '' }); set
Search(''); }}
      >
        × Clear
      </button>
    )}
  </div>
);
}

// — Board Column —————
function BoardColumn({ title, status, icon, tasks, onEdit, onDelete, onStatusChange }) {
  const filtered = tasks.filter(t => t.status === status);

```

```

return (
  <div className={`task-column ${status}`}>
    <div className="task-column__header">
      <span className="task-column__title">
        {icon} {title}
      </span>
      <span className="task-column__count">{filtered.length}</span>
    </div>
    {filtered.length === 0 ? (
      <div style={{ textAlign: 'center', color: 'var(--text-muted)', padding: '24px 0', fontSize: '0.825rem' }}>
        No tasks here
      </div>
    ) : (
      filtered.map(task => (
        <TaskCard
          key={task._id}
          task={task}
          onEdit={onEdit}
          onDelete={onDelete}
          onStatusChange={onStatusChange}
        />
      ))
    )}
  </div>
);
}

// — List Row —————
function ListRow({ task, onEdit, onDelete, onStatusChange }) {
  const overdue = isOverdue(task.dueDate, task.status);
  return (
    <div className={`task-list-row ${overdue ? 'overdue' : ''} ${task.status === 'completed' ? 'completed-row' : ''}`}>
      <span style={{ fontSize: '1.1rem' }}>
        {task.status === 'completed' ? '✅' : task.status === 'in-progress' ? '🔄' : '🚧'}
      </span>

      <div className="task-list-row__title">{task.title}</div>

      <div className="task-list-row__meta">
        <span className={`badge badge-priority-${task.priority}`}>{task.priority}</span>
        {task.category && task.category !== 'General' && (
          <span className="badge badge-category">{task.category}</span>
        )}
        {task.dueDate && (
          <span style={{ fontSize: '0.78rem', color: overdue ? 'var(--danger)' : 'var(--text-muted)' }}>
            📅 {formatDate(task.dueDate)}
          </span>
        )}
      </div>
    </div>
  );
}

```

```

    ))
    {task.assignedTo && (
      <span style={{ fontSize: '0.78rem', color: 'var(--text-muted)' }}>
         {task.assignedTo}
      </span>
    ))
    <select
      className="status-select"
      value={task.status}
      onChange={(e) => onStatusChange(task._id, e.target.value)}
    >
      <option value="todo">To Do</option>
      <option value="in-progress">In Progress</option>
      <option value="completed">Done</option>
    </select>
  </div>

  <div className="task-list-row__actions">
    <button className="btn btn-icon btn-sm" onClick={() => onEdit(task)} title="Edit">✎</button>
    <button className="btn btn-icon btn-sm" onClick={() => onDelete(task._id)} title="Delete" style={{ color: 'var(--danger)' }}>🗑️</button>
  </div>
</div>
);
}

// ——— Pagination ———
function Pagination({ pagination, onPageChange }) {
  const { currentPage, totalPages, totalCount, limit } = pagination;
  if (totalPages <= 1) return null;

  const start = (currentPage - 1) * limit + 1;
  const end = Math.min(currentPage * limit, totalCount);

  return (
    <div className="pagination">
      <button className="pagination__btn" onClick={() => onPageChange(1)} disabled={currentPage === 1}><<</button>
      <button className="pagination__btn" onClick={() => onPageChange(currentPage - 1)} disabled={currentPage === 1}><<</button>

      <span className="pagination__info">Page {currentPage} of {totalPages} ({start}–{end}) of {totalCount}</span>

      <button className="pagination__btn" onClick={() => onPageChange(currentPage + 1)} disabled={currentPage === totalPages}>>></button>
      <button className="pagination__btn" onClick={() => onPageChange(totalPages)} disabled={currentPage === totalPages}>>></button>
    </div>
  );
}

```

```

}

// — Main TaskList Component —————
function TaskList({
  tasks, loading, filters, onFilterChange,
  onEdit, onDelete, onStatusChange, onNewTask,
  pagination, onPageChange, activeView
}) {
  if (loading) {
    return (
      <div>
        <FilterBar filters={filters} onFilterChange={onFilterChange} />
        <div className="loading-wrapper">
          <div className="spinner"></div>
          <span>Loading tasks...</span>
        </div>
      </div>
    );
  }

  return (
    <div>
      <FilterBar filters={filters} onFilterChange={onFilterChange} />

      <div className="section-header">
        <h2>
          {filters.status
            ? `${filters.status === 'todo' ? '📌 To Do' : filters.status === 'in-progress' ? '🔄 In Progress' : '✅ Completed'} Tasks`
            : '📋 All Tasks'}
        </h2>
        <span className="section-header__count">{pagination.totalCount} task{pagination.totalCount !== 1 ? 's' : ''}</span>
      </div>

      {tasks.length === 0 ? (
        <div className="empty-state">
          <div className="empty-state__icon">📌</div>
          <h3 className="empty-state__title">No tasks found</h3>
          <p className="empty-state__desc">
            {filters.search || filters.status || filters.priority
              ? 'Try adjusting your filters'
              : 'Create your first task to get started!'}
          </p>
          <div>
            {filters.search && !filters.status && !filters.priority && (
              <button className="btn btn-primary" onClick={onNewTask}>
                + Create First Task
              </button>
            )}
          </div>
        </div>
      ) : activeView === 'board' ? (

```

```

    <div className="task-board">
      <BoardColumn title="To Do" status="todo" icon="📝" tasks={tasks} onEdit={onEdit}
onDelete={onDelete} onStatusChange={onStatusChange} />
      <BoardColumn title="In Progress" status="in-progress" icon="🔄" tasks={tasks}
onEdit={onEdit} onDelete={onDelete} onStatusChange={onStatusChange} />
      <BoardColumn title="Completed" status="completed" icon="✅" tasks={tasks} onEdit={onEdit}
onDelete={onDelete} onStatusChange={onStatusChange} />
    </div>
  ) : (
    <div className="task-list-view">
      {tasks.map(task => (
        <ListRow key={task._id} task={task} onEdit={onEdit} onDelete={onDelete} onStatusChange={onStatusChange} />
      ))}
    </div>
  )}

  <Pagination pagination={pagination} onPageChange={onPageChange} />
</div>
);
}

export default TaskList;

```

```

import React from 'react';

const PRIORITY_LABELS = { low: '🟢 Low', medium: '🟡 Medium', high: '🔴 High' };

function formatDate(dateStr) {
  if (!dateStr) return null;
  const d = new Date(dateStr);
  return d.toLocaleDateString('en-IN', { day: '2-digit', month: 'short', year: 'numeric'
});
}

function isOverdue(dueDate, status) {
  if (!dueDate || status === 'completed') return false;
  return new Date() > new Date(dueDate);
}

function TaskCard({ task, onEdit, onDelete, onStatusChange }) {
  const overdue = isOverdue(task.dueDate, task.status);

  return (
    <div className={`task-card ${overdue ? 'overdue' : ''} ${task.status === 'completed'
? 'completed-card' : ''}`}>
      <div className="task-card__header">
        <h3 className="task-card__title">{task.title}</h3>
        <div className="task-card__actions">
          <button
            className="btn btn-icon btn-sm"
            onClick={() => onEdit(task)}
            title="Edit task"
          >
            ✎
          </button>
          <button
            className="btn btn-icon btn-sm"
            onClick={() => onDelete(task._id)}
            title="Delete task"
            style={{ color: 'var(--danger)' }}
          >
            🗑
          </button>
        </div>
      </div>

      {task.description && (
        <p className="task-card__description">{task.description}</p>
      )}

      <div className="task-card__meta">

```

```

    <span className={`badge badge-priority-${task.priority}`}>
      {PRIORITY_LABELS[task.priority]}
    </span>
    {task.category && task.category !== 'General' && (
      <span className="badge badge-category">📁 {task.category}</span>
    )}
    {overdue && <span className="badge badge-overdue">⚠️ Overdue</span>}
    {task.tags && task.tags.slice(0, 2).map((tag, i) => (
      <span key={i} className="badge badge-tag">#{tag}</span>
    ))}
    {task.tags && task.tags.length > 2 && (
      <span className="badge badge-tag">+{task.tags.length - 2}</span>
    )}
  </div>

  <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between',
marginTop: '12px' }}>
    {/* Inline Status Change */}
    <select
      className="status-select"
      value={task.status}
      onChange={(e) => onStatusChange(task._id, e.target.value)}
    >
      <option value="todo">📌 To Do</option>
      <option value="in-progress">🔄 In Progress</option>
      <option value="completed">✅ Done</option>
    </select>

    {task.dueDate && (
      <span className={`task-card__due ${overdue ? 'overdue' : ''}`}>
        📅 {formatDate(task.dueDate)}
      </span>
    )}
  </div>

  {task.assignedTo && (
    <div style={{ marginTop: '8px', fontSize: '0.76rem', color: 'var(--text-muted)',
display: 'flex', alignItems: 'center', gap: '4px' }}>
      👤 {task.assignedTo}
    </div>
  )}
</div>
);
}

export default TaskCard;

```

```
import axios from 'axios';

const API_BASE_URL = process.env.REACT_APP_API_URL || 'http://localhost:5000/api';

const authAxios = axios.create({
  baseURL: API_BASE_URL,
  timeout: 15000,
  headers: { 'Content-Type': 'application/json' },
});

export const authAPI = {
  register: (data) => authAxios.post('/auth/register', data),
  login: (data) => authAxios.post('/auth/login', data),
  me: (token) => authAxios.get('/auth/me', {
    headers: { Authorization: `Bearer ${token}` }
  }),
};

export default authAxios;
```

```
import axios from 'axios';

const API_BASE_URL = process.env.REACT_APP_API_URL || 'http://localhost:5000/api';

const api = axios.create({
  baseURL: API_BASE_URL,
  timeout: 15000,
  headers: { 'Content-Type': 'application/json' },
});

api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('todo_token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

api.interceptors.response.use(
  res => res,
  err => {
    if (err.response?.status === 401) {
      localStorage.removeItem('todo_token');
      localStorage.removeItem('todo_user');
      if (window.location.pathname !== '/login') {
        window.location.href = '/login';
      }
    }
    return Promise.reject(new Error(err.response?.data?.message || err.message || 'Something went wrong'));
  }
);

export const habitAPI = {
  getAll: () => api.get('/habits'),
  create: (data) => api.post('/habits', data),
  update: (id, data) => api.put(`/habits/${id}`, data),
  delete: (id) => api.delete(`/habits/${id}`),
};

export const completionAPI = {
  getByMonth: (month) => api.get('/completions', { params: { month } }),
  getChart: (days = 30) => api.get('/completions/chart', { params: { days } }),
  toggle: (taskId, date) => api.post('/completions/toggle', { taskId, date }),
};
```

```
export default api;
```

```
import axios from 'axios';

// — API Base Configuration —————
const API_BASE_URL = process.env.REACT_APP_API_URL || 'http://localhost:5000/api';

const api = axios.create({
  baseURL: API_BASE_URL,
  timeout: 15000,
  headers: {
    'Content-Type': 'application/json',
  },
});

// — Request Interceptor —————
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('todo_token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// — Response Interceptor —————
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('todo_token');
      localStorage.removeItem('todo_user');
      if (window.location.pathname !== '/login') {
        window.location.href = '/login';
      }
    }
    const message =
      error.response?.data?.message ||
      error.message ||
      'Something went wrong. Please try again.';
    return Promise.reject(new Error(message));
  }
);

// — Task API Calls —————
export const taskAPI = {
  // READ ALL – with optional filters
  getAll: (params = {}) => api.get('/tasks', { params }),
}
```

```
// READ ONE
getById: (id) => api.get(`/tasks/${id}`),

// CREATE
create: (data) => api.post('/tasks', data),

// UPDATE (full)
update: (id, data) => api.put(`/tasks/${id}`, data),

// PATCH (partial)
patch: (id, data) => api.patch(`/tasks/${id}`, data),

// DELETE ONE
delete: (id) => api.delete(`/tasks/${id}`),

// DELETE ALL
deleteAll: () => api.delete('/tasks'),

// STATS
getStats: () => api.get('/tasks/stats'),
};

// — User API Calls —————
export const userAPI = {
  getAll: () => api.get('/users'),
  getById: (id) => api.get(`/users/${id}`),
  create: (data) => api.post('/users', data),
  update: (id, data) => api.put(`/users/${id}`, data),
  delete: (id) => api.delete(`/users/${id}`),
};

export default api;
```

```
/* — App Layout — */
.app { min-height: 100vh; display: flex; flex-direction: column; background: var(--bg); }
.app__main { flex: 1; max-width: 1600px; width: 100%; margin: 0 auto; padding: 24px 20px 60px; }

/* — Navbar — */
.navbar {
  background: var(--surface);
  border-bottom: 1px solid var(--border);
  position: sticky; top: 0; z-index: 100;
  box-shadow: 0 2px 20px rgba(0,0,0,0.4);
}
.navbar__inner {
  max-width: 1600px; margin: 0 auto; padding: 0 20px;
  height: 60px; display: flex; align-items: center; justify-content: space-between; gap: 20px;
}
.navbar__brand {
  display: flex; align-items: center; gap: 10px;
  font-size: 1.1rem; font-weight: 800; letter-spacing: 2px;
  color: var(--primary); white-space: nowrap;
}
.navbar__brand-icon {
  width: 34px; height: 34px; background: var(--primary-dark);
  border-radius: 8px; display: flex; align-items: center; justify-content: center;
  color: white; font-size: 1rem;
}
.navbar__tabs { display: flex; gap: 4px; }
.navbar__tab {
  padding: 7px 18px; border-radius: 8px; font-size: 0.85rem; font-weight: 600;
  background: transparent; color: var(--text-muted); transition: all 0.15s;
}
.navbar__tab:hover { background: var(--surface-2); color: var(--text); }
.navbar__tab.active { background: var(--primary-light); color: var(--primary); }
.navbar__actions { display: flex; align-items: center; gap: 10px; }

/* — Buttons — */
.btn {
  display: inline-flex; align-items: center; justify-content: center; gap: 6px;
  padding: 8px 16px; border-radius: 8px; font-size: 0.875rem; font-weight: 600;
  transition: all 0.15s ease; white-space: nowrap;
}
.btn-primary { background: var(--primary-dark); color: white; }
.btn-primary:hover { background: var(--primary); transform: translateY(-1px); box-shadow: 0 4px 12px rgba(99,102,241,0.4); }
.btn-secondary { background: var(--surface-2); color: var(--text); border: 1px solid var(--border); }
```

```

.btn-secondary:hover { background: var(--surface-3); }
.btn-danger { background: rgba(248,113,113,0.1); color: var(--danger); border: 1px solid
rgba(248,113,113,0.3); }
.btn-danger:hover { background: var(--danger); color: white; }
.btn-sm { padding: 5px 10px; font-size: 0.8rem; }
.btn-icon { padding: 7px; border-radius: 8px; background: var(--surface-2); color: var(-
-text-muted); border: 1px solid var(--border); }
.btn-icon:hover { background: var(--primary-light); color: var(--primary); }

/* — Dashboard ————— */
.dashboard { margin-bottom: 24px; }
.dashboard__grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(170px,
1fr)); gap: 14px; }
.stat-card {
  background: var(--surface); border: 1px solid var(--border); border-radius: var(--radi
us);
  padding: 18px; display: flex; align-items: center; gap: 14px;
  box-shadow: var(--shadow); transition: transform 0.15s, box-shadow 0.15s;
}
.stat-card:hover { transform: translateY(-2px); box-shadow: var(--shadow-md); }
.stat-card__icon { width: 44px; height: 44px; border-radius: 10px; display: flex; align-
items: center; justify-content: center; font-size: 1.3rem; flex-shrink: 0; }
.stat-card__value { font-size: 1.6rem; font-weight: 800; line-height: 1; color: var(--te
xt); }
.stat-card__label { font-size: 0.78rem; color: var(--text-muted); font-weight: 500; marg
in-top: 4px; }

/* — Filter Bar ————— */
.filter-bar {
  background: var(--surface); border: 1px solid var(--border); border-radius: var(--radi
us);
  padding: 14px 16px; margin-bottom: 18px; display: flex; flex-wrap: wrap; gap: 10px; al
ign-items: center;
}
.filter-bar__search { flex: 1; min-width: 200px; position: relative; }
.filter-bar__search input {
  width: 100%; padding: 8px 12px 8px 36px; border: 1px solid var(--border);
  border-radius: 8px; font-size: 0.875rem; background: var(--surface-2); color: var(--te
xt); transition: border-color 0.15s;
}
.filter-bar__search input:focus { border-color: var(--primary); }
.filter-bar__search input::placeholder { color: var(--text-dim); }
.filter-bar__search-icon { position: absolute; left: 11px; top: 50%; transform: translat
eY(-50%); color: var(--text-muted); font-size: 0.85rem; }
.filter-bar select {
  padding: 8px 12px; border: 1px solid var(--border); border-radius: 8px; font-size: 0.8
75rem;
  background: var(--surface-2); color: var(--text); cursor: pointer;
}
.filter-bar select:focus { border-color: var(--primary); }

```

```

/* — Task Board — */
.task-board { display: grid; grid-template-columns: repeat(3, 1fr); gap: 18px; align-items: start; }
.task-column { background: var(--surface); border: 1px solid var(--border); border-radius: var(--radius); padding: 14px; }
.task-column_header { display: flex; align-items: center; justify-content: space-between; margin-bottom: 14px; padding-bottom: 10px; border-bottom: 2px solid var(--border); }
.task-column_title { font-size: 0.82rem; font-weight: 700; display: flex; align-items: center; gap: 6px; text-transform: uppercase; letter-spacing: 0.5px; }
.task-column_count { background: var(--surface-2); color: var(--text-muted); font-size: 0.72rem; font-weight: 700; padding: 2px 8px; border-radius: 999px; }
.task-column_todo .task-column_title { color: #94a3b8; }
.task-column_in-progress .task-column_title { color: var(--secondary); }
.task-column_completed .task-column_title { color: var(--success); }

/* — Task Cards — */
.task-card {
  background: var(--surface-2); border: 1px solid var(--border); border-radius: var(--radius-sm);
  padding: 14px; margin-bottom: 10px; box-shadow: var(--shadow); transition: all 0.15s;
}
.task-card:hover { box-shadow: var(--shadow-md); transform: translateY(-1px); border-color: var(--border-bright); }
.task-card_overdue { border-left: 3px solid var(--danger); }
.task-card_completed-card { opacity: 0.6; }
.task-card_header { display: flex; align-items: flex-start; justify-content: space-between; gap: 8px; margin-bottom: 8px; }
.task-card_title { font-size: 0.9rem; font-weight: 600; color: var(--text); line-height: 1.4; word-break: break-word; }
.task-card_completed-card .task-card_title { text-decoration: line-through; color: var(--text-muted); }
.task-card_actions { display: flex; gap: 3px; flex-shrink: 0; opacity: 0; transition: opacity 0.15s; }
.task-card:hover .task-card_actions { opacity: 1; }
.task-card_description { font-size: 0.8rem; color: var(--text-muted); margin-bottom: 10px; line-height: 1.5; display: -webkit-box; -webkit-line-clamp: 2; -webkit-box-orient: vertical; overflow: hidden; }
.task-card_meta { display: flex; flex-wrap: wrap; gap: 5px; align-items: center; }
.task-card_due { font-size: 0.72rem; color: var(--text-muted); margin-top: 8px; display: flex; align-items: center; gap: 3px; }
.task-card_due_overdue { color: var(--danger); font-weight: 600; }

/* — Badges — */
.badge { display: inline-flex; align-items: center; gap: 3px; padding: 2px 7px; border-radius: 999px; font-size: 0.68rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.3px; }
.badge_priority-low { background: rgba(52,211,153,0.1); color: #34d399; border: 1px solid rgba(52,211,153,0.3); }
.badge_priority-medium { background: rgba(251,191,36,0.1); color: #fbbf24; border: 1px solid rgba(251,191,36,0.3); }
.badge_priority-high { background: rgba(248,113,113,0.1); color: #f87171; border: 1px solid

```

```

lid rgba(248,113,113,0.3); }
.badge-status-todo { background: var(--surface-2); color: var(--text-muted); border: 1px
solid var(--border); }
.badge-status-in-progress { background: rgba(56,189,248,0.1); color: #38bdf8; border: 1p
x solid rgba(56,189,248,0.3); }
.badge-status-completed { background: rgba(52,211,153,0.1); color: #34d399; border: 1px
solid rgba(52,211,153,0.3); }
.badge-category { background: var(--primary-light); color: var(--primary); border: 1px s
olid rgba(129,140,248,0.3); }
.badge-overdue { background: rgba(248,113,113,0.1); color: var(--danger); border: 1px so
lid rgba(248,113,113,0.3); }
.badge-tag { background: var(--surface-2); color: var(--text-muted); border: 1px solid v
ar(--border); }

/* — Status Select ————— */
.status-select { padding: 3px 7px; border-radius: 6px; border: 1px solid var(--border);
font-size: 0.72rem; font-weight: 600; cursor: pointer; background: var(--surface-2); col
or: var(--text); }

/* — Task List View ————— */
.task-list-view { display: flex; flex-direction: column; gap: 8px; }
.task-list-row {
  background: var(--surface); border: 1px solid var(--border); border-radius: var(--radi
us-sm);
  padding: 12px 16px; display: flex; align-items: center; gap: 12px; transition: all 0.1
5s;
}
.task-list-row:hover { border-color: var(--primary); box-shadow: var(--shadow-md); }
.task-list-row.overdue { border-left: 3px solid var(--danger); }
.task-list-row__title { flex: 1; font-size: 0.88rem; font-weight: 600; color: var(--tex
t); }
.task-list-row.completed-row .task-list-row__title { text-decoration: line-through; colo
r: var(--text-muted); }
.task-list-row__meta { display: flex; align-items: center; gap: 8px; flex-wrap: wrap; }
.task-list-row__actions { display: flex; gap: 4px; }

/* — Modal ————— */
.modal-overlay {
  position: fixed; inset: 0; background: rgba(0,0,0,0.7); backdrop-filter: blur(6px);
  z-index: 200; display: flex; align-items: center; justify-content: center; padding: 20
px;
  animation: fadeIn 0.15s ease;
}
@keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }
@keyframes slideUp { from { opacity: 0; transform: translateY(20px); } to { opacity: 1;
transform: translateY(0); } }

.modal {
  background: var(--surface); border: 1px solid var(--border); border-radius: var(--radi
us-lg);
  width: 100%; max-width: 540px; max-height: 90vh; overflow-y: auto;

```

```

    box-shadow: var(--shadow-lg); animation: slideUp 0.2s ease;
}
.modal__header { display: flex; align-items: center; justify-content: space-between; padding: 22px 24px 0; }
.modal__title { font-size: 1.15rem; font-weight: 700; color: var(--text); }
.modal__close { width: 30px; height: 30px; border-radius: 7px; background: var(--surface-2); display: flex; align-items: center; justify-content: center; color: var(--text-muted); transition: all 0.15s; }
.modal__close:hover { background: rgba(248,113,113,0.15); color: var(--danger); }
.modal__body { padding: 18px 24px 24px; }

/* — Forms — */
.form-group { margin-bottom: 16px; }
.form-label { display: block; font-size: 0.8rem; font-weight: 600; color: var(--text-muted); margin-bottom: 6px; text-transform: uppercase; letter-spacing: 0.5px; }
.form-label span { color: var(--danger); margin-left: 2px; }
.form-control { width: 100%; padding: 9px 12px; border: 1.5px solid var(--border); border-radius: 8px; font-size: 0.875rem; color: var(--text); background: var(--surface-2); transition: border-color 0.15s, box-shadow 0.15s; }
.form-control:focus { border-color: var(--primary); box-shadow: 0 0 0 3px rgba(129,140,248,0.15); }
.form-control::placeholder { color: var(--text-dim); }
textarea.form-control { resize: vertical; min-height: 75px; }
.form-row { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; }
.form-actions { display: flex; justify-content: flex-end; gap: 10px; padding-top: 6px; }

/* — Type Toggle (habit form) — */
.type-toggle { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; }
.type-btn {
  padding: 12px; border-radius: 10px; border: 1.5px solid var(--border);
  background: var(--surface-2); color: var(--text-muted); text-align: center;
  display: flex; flex-direction: column; align-items: center; gap: 4px; transition: all 0.15s;
}
.type-btn small { font-size: 0.72rem; color: var(--text-dim); font-weight: 400; }
.type-btn:hover { border-color: var(--primary); color: var(--text); }
.type-btn.active { border-color: var(--primary); background: var(--primary-light); color: var(--primary); }
.type-btn.active small { color: var(--primary); opacity: 0.7; }

/* — Day Selector — */
.day-selector { display: flex; gap: 6px; flex-wrap: wrap; }
.day-btn { width: 44px; height: 44px; border-radius: 50%; border: 1.5px solid var(--border); background: var(--surface-2); color: var(--text-muted); font-size: 0.78rem; font-weight: 600; transition: all 0.15s; }
.day-btn:hover { border-color: var(--primary); color: var(--text); }
.day-btn.active { border-color: var(--primary); background: var(--primary-dark); color: white; }

/* — Color Picker — */
.color-picker { display: flex; gap: 8px; flex-wrap: wrap; }

```

```

.color-dot { width: 28px; height: 28px; border-radius: 50%; border: 2px solid transparent; transition: all 0.15s; }
.color-dot.active { border-color: white; transform: scale(1.2); box-shadow: 0 0 0 2px var(--surface-2); }
.color-dot:hover { transform: scale(1.15); }

/* — Progress Chart ————— */
.chart-wrap {
  background: var(--surface); border: 1px solid var(--border); border-radius: var(--radius);
  padding: 20px; margin-bottom: 24px; box-shadow: var(--shadow);
}
.chart-header { display: flex; align-items: flex-start; justify-content: space-between; margin-bottom: 16px; }
.chart-title { font-size: 1rem; font-weight: 700; color: var(--text); }
.chart-sub { font-size: 0.8rem; color: var(--text-muted); margin-top: 2px; }
.chart-badges { display: flex; gap: 6px; align-items: center; flex-wrap: wrap; }
.chart-badge { background: var(--surface-2); color: var(--text-dim); font-size: 0.72rem; padding: 2px 8px; border-radius: 999px; }
.chart-svg-wrap { width: 100%; overflow: hidden; border-radius: 8px; }
.chart-svg { width: 100%; height: 180px; display: block; }
.chart-loading { display: flex; align-items: center; gap: 12px; color: var(--text-muted); min-height: 180px; justify-content: center; }

/* — Habit Tracker ————— */
.habit-tracker { width: 100%; }
.ht-header { display: flex; align-items: center; justify-content: space-between; margin-bottom: 16px; }
.ht-month-nav { display: flex; align-items: center; gap: 12px; }
.ht-month-title { font-size: 1.1rem; font-weight: 700; color: var(--text); min-width: 160px; text-align: center; }
.ht-nav-btn { width: 32px; height: 32px; border-radius: 8px; background: var(--surface); border: 1px solid var(--border); color: var(--text); font-size: 1.1rem; display: flex; align-items: center; justify-content: center; transition: all 0.15s; }
.ht-nav-btn:hover { background: var(--surface-2); border-color: var(--primary); color: var(--primary); }

/* Grid wrapper - scrollable */
.ht-grid-wrap { width: 100%; overflow-x: auto; border-radius: var(--radius); border: 1px solid var(--border); background: var(--surface); box-shadow: var(--shadow); }

.ht-grid { border-collapse: collapse; width: max-content; min-width: 100%; }

/* Header row */
.ht-th-task { position: sticky; left: 0; z-index: 10; background: var(--surface); min-width: 220px; max-width: 260px; padding: 10px 14px; text-align: left; border-bottom: 1px solid var(--border); border-right: 1px solid var(--border); }
.ht-th-pct { width: 60px; min-width: 60px; padding: 10px 8px; text-align: center; border-bottom: 1px solid var(--border); border-right: 1px solid var(--border); background: var(--surface); color: var(--text-muted); font-size: 0.72rem; }
.ht-th-day { min-width: 36px; width: 36px; padding: 6px 2px; text-align: center; border-

```

```

bottom: 1px solid var(--border); border-right: 1px solid rgba(51,65,85,0.5); }
.ht-today-col { background: rgba(129,140,248,0.08) !important; }
.ht-day-label { display: flex; flex-direction: column; align-items: center; gap: 1px; }
.ht-day-num { font-size: 0.78rem; font-weight: 700; color: var(--text); }
.ht-day-abbr { font-size: 0.62rem; color: var(--text-dim); }

/* Day pct row */
.ht-day-pct-row td { background: rgba(15,23,42,0.5); border-bottom: 2px solid var(--border); padding: 4px 2px; }
.ht-td-daypct { text-align: center; font-size: 0.62rem; font-weight: 700; }
.ht-daypct-val { display: block; }
.ht-day-pct-label { font-size: 0.68rem; color: var(--text-dim); font-style: italic; padding-left: 14px; }

/* Data rows */
.ht-row { border-bottom: 1px solid rgba(51,65,85,0.5); transition: background 0.1s; }
.ht-row:hover { background: rgba(129,140,248,0.04); }
.ht-row:last-child { border-bottom: none; }

.ht-td-label {
  position: sticky; left: 0; z-index: 5; background: var(--surface);
  padding: 8px 12px; min-width: 220px; max-width: 260px;
  border-right: 1px solid var(--border);
  display: flex; align-items: center; justify-content: space-between;
}
.ht-row:hover .ht-td-label { background: #1a2540; }

.ht-task-info { display: flex; align-items: center; gap: 7px; min-width: 0; }
.ht-task-dot { width: 8px; height: 8px; border-radius: 50%; flex-shrink: 0; }
.ht-task-name { font-size: 0.82rem; font-weight: 600; white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
.ht-type-badge { font-size: 0.7rem; flex-shrink: 0; }

.ht-task-actions { display: flex; gap: 3px; opacity: 0; transition: opacity 0.15s; flex-shrink: 0; }
.ht-row:hover .ht-task-actions { opacity: 1; }
.ht-action-btn { background: none; border: none; font-size: 0.75rem; padding: 2px 3px; border-radius: 4px; color: var(--text-muted); transition: background 0.1s; }
.ht-action-btn:hover { background: var(--surface-2); }

/* % column */
.ht-td-pct { width: 60px; min-width: 60px; padding: 8px 6px; border-right: 1px solid var(--border); text-align: center; vertical-align: middle; }
.ht-pct-wrap { display: flex; flex-direction: column; align-items: center; gap: 4px; }
.ht-pct-val { font-size: 0.72rem; font-weight: 700; }
.ht-pct-bar { width: 40px; height: 3px; background: var(--surface-2); border-radius: 999px; overflow: hidden; }
.ht-pct-fill { height: 100%; border-radius: 999px; transition: width 0.3s; }

/* Checkboxes */
.ht-td-cell { padding: 6px 2px; text-align: center; vertical-align: middle; border-right:

```

```

t: 1px solid rgba(51,65,85,0.3); }
.ht-checkbox {
  width: 24px; height: 24px; border-radius: 5px; border: 1.5px solid;
  background: transparent; display: inline-flex; align-items: center; justify-content: center;
  cursor: pointer; transition: all 0.12s; font-size: 0.8rem;
}
.ht-checkbox:hover:not(:disabled) { transform: scale(1.15); }
.ht-checkbox.checked { }
.ht-checkbox.future { opacity: 0.3; cursor: not-allowed; }
.ht-check { font-weight: 800; line-height: 1; }
.ht-na { color: var(--text-dim); font-size: 1rem; line-height: 24px; }

/* Legend */
.ht-legend {
  display: flex; flex-wrap: wrap; gap: 16px; margin-top: 14px;
  padding: 12px 16px; background: var(--surface); border: 1px solid var(--border);
  border-radius: 8px; font-size: 0.78rem; color: var(--text-muted);
}
.ht-legend-item { display: flex; align-items: center; gap: 6px; }
.ht-legend-dot { width: 10px; height: 10px; border-radius: 50%; flex-shrink: 0; }

/* — Section Header — */
.section-header { display: flex; align-items: center; justify-content: space-between; margin-bottom: 14px; }
.section-header h2 { font-size: 1rem; font-weight: 700; color: var(--text); }
.section-header__count { font-size: 0.8rem; color: var(--text-muted); background: var(--surface); border: 1px solid var(--border); padding: 3px 10px; border-radius: 999px; }

/* — Empty State — */
.empty-state { text-align: center; padding: 60px 20px; color: var(--text-muted); }
.empty-state__icon { font-size: 3rem; margin-bottom: 14px; }
.empty-state__title { font-size: 1.05rem; font-weight: 600; color: var(--text); margin-bottom: 8px; }
.empty-state__desc { font-size: 0.875rem; margin-bottom: 18px; }

/* — Loading — */
.loading-wrapper { display: flex; flex-direction: column; align-items: center; justify-content: center; padding: 60px; gap: 14px; color: var(--text-muted); }
.spinner { width: 36px; height: 36px; border: 3px solid var(--surface-2); border-top-color: var(--primary); border-radius: 50%; animation: spin 0.8s linear infinite; }
@keyframes spin { to { transform: rotate(360deg); } }

/* — Pagination — */
.pagination { display: flex; align-items: center; justify-content: center; gap: 6px; margin-top: 20px; }
.pagination__btn { width: 34px; height: 34px; border-radius: 7px; background: var(--surface); border: 1px solid var(--border); color: var(--text); font-size: 0.875rem; display: flex; align-items: center; justify-content: center; transition: all 0.15s; }
.pagination__btn:hover:not(:disabled) { background: var(--primary-dark); color: white; border-color: var(--primary-dark); }

```

```

.pagination__btn.active { background: var(--primary-dark); color: white; border-color: v
ar(--primary-dark); }
.pagination__btn.disabled { opacity: 0.35; cursor: not-allowed; }
.pagination__info { font-size: 0.8rem; color: var(--text-muted); margin: 0 6px; }

/* — Responsive — */
@media (max-width: 900px) {
  .task-board { grid-template-columns: 1fr; }
  .form-row { grid-template-columns: 1fr; }
}
@media (max-width: 600px) {
  .app__main { padding: 14px 12px 40px; }
  .dashboard__grid { grid-template-columns: repeat(2, 1fr); }
  .navbar__brand-text { font-size: 0.85rem; letter-spacing: 1px; }
  .navbar__user-name { display: none; }
}

/* — Navbar User & Logout — */
.navbar__user {
  display: flex;
  align-items: center;
  gap: 10px;
  background: var(--surface-2);
  padding: 4px 10px 4px 6px;
  border-radius: 999px;
  border: 1px solid var(--border);
}
.navbar__user-avatar {
  width: 28px;
  height: 28px;
  border-radius: 50%;
  background: linear-gradient(135deg, #6366f1, #a855f7);
  color: white;
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: 700;
  font-size: 0.8rem;
}
.navbar__user-name {
  font-size: 0.85rem;
  font-weight: 600;
  color: var(--text);
  max-width: 120px;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
.navbar__logout-btn {
  padding: 4px 10px !important;
  font-size: 0.75rem !important;
}

```

```

    border-radius: 6px !important;
    border-color: rgba(239, 68, 68, 0.4) !important;
    color: #fca5a5 !important;
    background: transparent !important;
}
.navbar__logout-btn:hover {
    background: rgba(239, 68, 68, 0.15) !important;
    border-color: #ef4444 !important;
    color: #fff !important;
}

/* — Auth Page Styles ————— */
.auth-page {
    min-height: 100vh;
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 20px;
    background: #0b1120;
    position: relative;
    overflow: hidden;
    font-family: 'Inter', system-ui, -apple-system, sans-serif;
}

.auth-bg {
    position: absolute;
    inset: 0;
    pointer-events: none;
    overflow: hidden;
}

.auth-blob {
    position: absolute;
    border-radius: 50%;
    filter: blur(100px);
    opacity: 0.4;
    animation: pulse 8s ease-in-out infinite alternate;
}

.auth-blob-1 {
    width: 400px;
    height: 400px;
    top: -100px;
    left: -50px;
    background: radial-gradient(circle, #6366f1 0%, transparent 70%);
}

.auth-blob-2 {
    width: 450px;
    height: 450px;
    bottom: -100px;

```

```
    right: -50px;
    background: radial-gradient(circle, #06b6d4 0%, transparent 70%);
    animation-delay: 4s;
}

@keyframes pulse {
    0% { transform: scale(1) translate(0, 0); }
    100% { transform: scale(1.15) translate(20px, 30px); }
}

.auth-card {
    position: relative;
    z-index: 10;
    width: 100%;
    max-width: 440px;
    background: rgba(30, 41, 59, 0.85);
    backdrop-filter: blur(16px);
    -webkit-backdrop-filter: blur(16px);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 20px;
    padding: 36px 32px;
    box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.5), 0 0 30px rgba(99, 102, 241, 0.15);
    text-align: center;
}

.auth-logo {
    display: inline-flex;
    align-items: center;
    gap: 10px;
    margin-bottom: 6px;
}

.auth-logo-icon {
    width: 36px;
    height: 36px;
    border-radius: 10px;
    background: linear-gradient(135deg, #6366f1, #06b6d4);
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 1.25rem;
    font-weight: 800;
    box-shadow: 0 0 16px rgba(99, 102, 241, 0.4);
}

.auth-logo-text {
    font-size: 1.35rem;
    font-weight: 800;
    letter-spacing: 1.5px;
    background: linear-gradient(135deg, #e2e8f0 30%, #93c5fd 100%);
```

```
-webkit-background-clip: text;
-webkit-text-fill-color: transparent;
margin: 0;
}

.auth-tagline {
  font-size: 0.85rem;
  color: #94a3b8;
  margin: 0 0 24px 0;
}

.auth-tabs {
  display: grid;
  grid-template-columns: 1fr 1fr;
  background: rgba(15, 23, 42, 0.7);
  padding: 4px;
  border-radius: 12px;
  margin-bottom: 24px;
  border: 1px solid rgba(255, 255, 255, 0.05);
}

.auth-tab {
  padding: 9px;
  border: none;
  background: transparent;
  color: #94a3b8;
  font-weight: 600;
  font-size: 0.85rem;
  border-radius: 8px;
  cursor: pointer;
  transition: all 0.2s ease;
}

.auth-tab.active {
  background: #6366f1;
  color: #ffffff;
  box-shadow: 0 2px 8px rgba(99, 102, 241, 0.35);
}

.auth-form {
  display: flex;
  flex-direction: column;
  gap: 16px;
  text-align: left;
}

.auth-field {
  display: flex;
  flex-direction: column;
  gap: 6px;
}
```

```
.auth-label {
  font-size: 0.8rem;
  font-weight: 500;
  color: #cbd5e1;
}

.auth-input {
  width: 100%;
  padding: 11px 14px;
  border-radius: 10px;
  border: 1px solid rgba(255, 255, 255, 0.1);
  background: rgba(15, 23, 42, 0.6);
  color: #f8fafc;
  font-size: 0.9rem;
  transition: all 0.2s;
  outline: none;
  box-sizing: border-box;
}

.auth-input:focus {
  border-color: #6366f1;
  box-shadow: 0 0 3px rgba(99, 102, 241, 0.25);
  background: rgba(15, 23, 42, 0.8);
}

.auth-hint {
  font-size: 0.72rem;
  color: #64748b;
  margin-top: 2px;
}

.auth-error {
  background: rgba(239, 68, 68, 0.15);
  border: 1px solid rgba(239, 68, 68, 0.3);
  color: #fca5a5;
  padding: 10px 14px;
  border-radius: 8px;
  font-size: 0.82rem;
  text-align: left;
}

.auth-submit {
  margin-top: 6px;
  padding: 12px;
  border-radius: 10px;
  border: none;
  background: linear-gradient(135deg, #6366f1, #4f46e5);
  color: white;
  font-weight: 600;
  font-size: 0.92rem;
}
```

```
    cursor: pointer;
    transition: all 0.2s;
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 8px;
    box-shadow: 0 4px 14px rgba(99, 102, 241, 0.4);
}

.auth-submit:hover:not(:disabled) {
    opacity: 0.92;
    transform: translateY(-1px);
    box-shadow: 0 6px 20px rgba(99, 102, 241, 0.5);
}

.auth-submit:disabled {
    opacity: 0.6;
    cursor: not-allowed;
}

.auth-spinner {
    width: 16px;
    height: 16px;
    border: 2px solid rgba(255, 255, 255, 0.3);
    border-top-color: white;
    border-radius: 50%;
    animation: spin 0.6s linear infinite;
    display: inline-block;
}

.auth-switch {
    margin-top: 20px;
    font-size: 0.82rem;
    color: #94a3b8;
}

.auth-link {
    background: transparent;
    border: none;
    color: #818cf8;
    font-weight: 600;
    cursor: pointer;
    padding: 0;
    margin-left: 4px;
    text-decoration: underline;
}

.auth-link:hover {
    color: #a5b4fc;
}
```

```
.auth-features {  
  display: flex;  
  justify-content: center;  
  gap: 16px;  
  margin-top: 28px;  
  padding-top: 20px;  
  border-top: 1px solid rgba(255, 255, 255, 0.06);  
  font-size: 0.75rem;  
  color: #64748b;  
}
```

```
{
  "short_name": "TODO Tracker",
  "name": "TO-DO-TRACKER - Habit & Task Manager",
  "description": "Track habits. Hit goals. Build streaks with dark theme habit tracking and progress charts.",
  "icons": [
    {
      "src": "favicon.png",
      "sizes": "64x64",
      "type": "image/png"
    },
    {
      "src": "favicon.ico",
      "sizes": "64x64 32x32 24x24 16x16",
      "type": "image/x-icon"
    },
    {
      "src": "logo192.png",
      "type": "image/png",
      "sizes": "192x192",
      "purpose": "any maskable"
    },
    {
      "src": "logo512.png",
      "type": "image/png",
      "sizes": "512x512",
      "purpose": "any maskable"
    }
  ],
  "start_url": "/",
  "display": "standalone",
  "orientation": "portrait",
  "theme_color": "#0b1120",
  "background_color": "#0b1120"
}
```

```
{
  "name": "taskflow-frontend",
  "version": "1.0.0",
  "private": true,
  "dependencies": {
    "@testing-library/jest-dom": "^5.17.0",
    "@testing-library/react": "^13.4.0",
    "@testing-library/user-event": "^13.5.0",
    "axios": "^1.6.2",
    "lucide-react": "^0.294.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-scripts": "5.0.1",
    "react-toastify": "^9.1.3",
    "web-vitals": "^2.1.4"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject"
  },
  "eslintConfig": {
    "extends": ["react-app", "react-app/jest"],
    "rules": {
      "no-unused-vars": "warn"
    }
  },
  "browserslist": {
    "production": [">0.2%", "not dead", "not op_mini all"],
    "development": ["last 1 chrome version", "last 1 firefox version", "last 1 safari version"]
  },
  "proxy": "http://localhost:5000"
}
```

```
{
  "name": "taskflow-backend",
  "version": "1.0.0",
  "description": "TaskFlow - Cloud-Based Task Management System Backend",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "cors": "^2.8.5",
    "dotenv": "^16.3.1",
    "express": "^4.18.2",
    "express-validator": "^7.0.1",
    "helmet": "^7.1.0",
    "jsonwebtoken": "^9.0.3",
    "mongoose": "^8.0.3",
    "morgan": "^1.10.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.2"
  },
  "keywords": [
    "cloud",
    "taskmanagement",
    "nodejs",
    "mongodb"
  ],
  "author": "TaskFlow Team",
  "license": "MIT"
}
```